



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

EMOTION PROMPTING E ANALISI COGNITIVA APPLICATA ALL'INTELLIGENZA ARTIFICIALE

RELATORE

Prof. Fabio Palomba

Università degli Studi di Salerno

CANDIDATO

Pietro Coticella

Matricola: 0512118457

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

“Il nostro tempo è limitato, per cui non lo dobbiamo sprecare vivendo la vita di qualcun altro. Non facciamoci intrappolare dai dogmi, che vuol dire vivere seguendo i risultati del pensiero di altre persone. Non lasciamo che il rumore delle opinioni altrui offuschi la nostra voce interiore. E, cosa più importante di tutte, dobbiamo avere il coraggio di seguire il nostro cuore e la nostra intuizione. In qualche modo, essi sanno che cosa vogliamo realmente diventare. Tutto il resto è secondario.”

— Steve Jobs, Stanford Commencement Address, 2005

Abstract

Negli ultimi anni l'intelligenza artificiale ha assunto un ruolo sempre più rilevante nella vita quotidiana, influenzando in modo profondo il modo in cui comunichiamo, lavoriamo e prendiamo decisioni. I sistemi basati su modelli linguistici di grandi dimensioni (*Large Language Models*, LLM) sono ormai in grado di generare testi complessi, risolvere problemi e adattarsi al contesto conversazionale dell'utente. Tuttavia, tali modelli presentano ancora limiti nella capacità di interpretare in modo consapevole le componenti emotive e cognitive del linguaggio umano, poiché la loro comprensione si fonda su correlazioni linguistiche piuttosto che su un'elaborazione affettiva o cognitiva reale. Da questa problematica nasce la necessità di esplorare e valutare se un sistema di intelligenza artificiale sia effettivamente in grado di adattare il proprio comportamento in funzione dello stato emotivo e dello stile cognitivo dell'utente, modulando linguaggio, tono e contenuti generati in modo coerente con il profilo individuale. Il presente lavoro di tesi si inserisce in questo contesto, proponendo la progettazione e lo sviluppo di un sistema *AI-based* che integra le API di *OpenAI* e *Gemini* con gli strumenti di *Google Workspace* (*Moduli* e *Sheets*). L'obiettivo è quello di automatizzare la raccolta e l'analisi dei dati provenienti da questionari online, analizzando le risposte testuali attraverso tecniche di *emotion prompting*, *sentiment analysis* e analisi dello stile cognitivo. Il sistema è in grado di riconoscere il profilo cognitivo dell'utente (analitico o olistico) e di adattare di conseguenza il linguaggio, il tono e la struttura delle risposte fornite, generando inoltre due versioni distinte di codice Python — una oggettiva e una emotiva — accompagnate da un file *README* personalizzato. La generazione del codice Python rappresenta un esperimento mirato a valutare l'efficacia dell'*emotion prompting*. Attraverso la creazione di due versioni differenti — una neutra e una influenzata dall'emozione rilevata — è possibile analizzare come la presenza di componenti emotive nei prompt modifichi la struttura, il linguaggio e l'organizzazione logica del codice prodotto dall'intelligenza artificiale. Tale confronto consente di osservare in modo concreto se e in che misura l'AI sia sensibile agli stimoli emotivi inseriti nel prompt, e se tali variazioni si traducano in differenze qualitative nel risultato finale. Il file *README* generato dal sistema costituisce invece la parte sperimentale dedicata all'analisi cognitiva. Esso viene prodotto in due versioni, ciascuna adattata allo stile cognitivo dell'utente rilevato tramite il questionario elaborato da *Gemini*. Nel caso di un profilo analitico, il linguaggio del

README risulta schematico, strutturato e orientato ai dettagli tecnici; al contrario, per un profilo olistico, il tono diventa più discorsivo e contestuale, privilegiando la comprensione generale e la connessione tra concetti. Questa fase sperimentale consente di verificare se l'intelligenza artificiale sia in grado di modulare non solo il contenuto ma anche la modalità comunicativa in base alle caratteristiche cognitive dell'utente. Un'applicazione concreta di questo approccio può essere trovata in ambito educativo. Un docente, ad esempio, potrebbe utilizzare il sistema per analizzare le risposte di un questionario rivolto agli studenti, comprendendo se prevale uno stile cognitivo analitico o olistico. L'intelligenza artificiale, sulla base di tali informazioni, fornirebbe spiegazioni e materiali didattici differenziati: sintetici e schematici per gli utenti analitici, più narrativi e contestuali per quelli olistici. In tal modo, l'esperienza di apprendimento diventerebbe più efficace e coinvolgente. I risultati preliminari ottenuti mostrano che la combinazione tra analisi emotiva e cognitiva migliora la pertinenza e la coerenza delle risposte dell'AI, favorendo un'interazione più naturale e intuitiva tra utente e sistema. Inoltre, è emerso che l'inserimento di prompt emotivi all'interno delle richieste indirizzate al modello contribuisce a incrementare la qualità del codice generato, rendendolo più leggibile, coerente e contestualizzato rispetto all'intento dell'utente. Il modello è stato in grado di riconoscere correttamente i principali stati emotivi e i profili cognitivi, adattando il linguaggio, la struttura e la modalità di presentazione dei contenuti generati. Questi risultati confermano la validità dell'approccio proposto e aprono la strada a futuri sviluppi nel campo dell'intelligenza artificiale adattiva, orientata alla personalizzazione cognitivo-emotiva e alla comprensione del linguaggio umano in tutte le sue sfumature.

Indice

Elenco delle Figure	iii
Elenco delle Tabelle	v
1 Introduzione	1
1.1 Motivazioni ed obiettivi	1
1.2 Risultati ottenuti	3
1.3 Struttura della tesi	5
2 Background e stato dell'arte	6
2.1 Emotion prompting	6
2.2 Sentiment analysis nei LLM	9
2.3 Analisi dello stile cognitivo	11
3 Metodologia e applicazione sperimentale dell'<i>emotion prompting</i> e dell'analisi cognitiva nei sistemi di intelligenza artificiale	13
3.1 Obiettivi e domande di ricerca	13
3.2 Disegno metodologico generale	15
3.2.1 Fase 1 – Classificazione cognitiva (RQ1)	16
3.2.2 Fase 2 – Analisi emotiva del prompt (RQ2)	20

3.2.3	Fase 3 – Comparator per la valutazione della qualità del codice (RQ3)	25
3.2.4	Fase 4 – Integrazione dei moduli e generazione del README (RQ4)	30
4	Analisi dei risultati	36
4.1	Introduzione ai risultati	36
4.2	Risultati attesi RQ1: classificatore cognitivo	37
4.2.1	Struttura sperimentale.	38
4.3	Risultati attesi RQ2: accuratezza della sentiment analysis	42
4.3.1	Struttura sperimentale	42
4.3.2	Test 1: prompt arrabbiato	42
4.3.3	Test 2: prompt entusiasta	44
4.3.4	Confronto tra i due test	45
4.3.5	Conclusioni relative alla RQ2	45
4.4	Risultati attesi RQ3: efficacia dell’emotion prompting sul codice generato	46
4.4.1	Struttura del test	46
4.4.2	Confronto tra codice neutro e codice con Emotion Prompting	47
4.5	Risultati attesi RQ4: adattamento linguistico nella generazione del README	52
4.5.1	Struttura sperimentale	52
5	Conclusioni	59
	Bibliografia	62

Elenco delle figure

3.1	Schema generale del flusso metodologico del sistema sperimentale. .	16
3.2	Pipeline della Fase 1 – Raccolta e classificazione automatica dello stile cognitivo tramite le API di <i>Gemini</i>	19
3.3	Pipeline della Fase 2 – Analisi emotiva del prompt. Il diagramma mostra il flusso di elaborazione che porta dall’input utente alla generazione del profilo emotivo e delle due varianti del prompt (<i>oggettiva ed emotiva</i>), tramite le API di <i>OpenAI</i>	24
3.4	Pipeline della Fase 3 – Comparator per la valutazione della qualità del codice. La figura illustra il processo di confronto tra le due versioni di codice (generate da prompt neutro ed emotivo) e la valutazione automatica condotta dal modello <i>OpenAI</i> secondo criteri di correttezza, leggibilità e aderenza alle best practice.	28
3.5	Pipeline della Fase 4 – Integrazione dei moduli e generazione del README. La figura mostra l’interazione tra il modulo cognitivo (<i>Gemini</i>) e quello emotivo (<i>OpenAI</i>) e la successiva generazione automatica dei due output principali: il codice Python (versione neutra ed emotiva) e il file <code>README.md</code> adattato allo stile cognitivo dell’utente.	32
3.6	Schema generale del Processo di estrazione e analisi dei dati.	35

4.1	Codice Python generato con prompt neutro per la classe <code>ContoBancario</code> . Il codice risulta funzionale ma privo di commenti, validazioni avanzate e formattazione curata.	49
4.2	Codice Python generato con <i>Emotion Prompting</i> . La versione risulta più leggibile, documentata e aderente alle buone pratiche di programma- zione Python, con un uso coerente delle eccezioni e una formattazione più curata.	50
4.3	README generato con prompt neutro per la classe <code>ContoBancario</code> con stile cognitivo <code>olistico</code>	55

Elenco delle tabelle

3.1	Criteri di valutazione dell'accuratezza del classificatore cognitivo. . .	20
3.2	Criteri di valutazione utilizzati dal <i>comparator</i> per l'analisi della qualità del codice.	29
3.3	Relazione tra profilo cognitivo e caratteristiche linguistiche del file README.md.	34
4.1	Mini-dataset di test per la classificazione dello stile cognitivo tramite <i>Gemini</i>	40
4.2	Confronto sintetico tra i risultati dei due test di <i>sentiment analysis</i> . . .	45
4.3	Confronto sintetico tra i due file README.md generati.	58

CAPITOLO 1

Introduzione

1.1 Motivazioni ed obiettivi

I *Large Language Models* (LLM) hanno rivoluzionato il modo in cui le persone interagiscono con la tecnologia, consentendo un dialogo più naturale e fluido tra utente e macchina. Questa evoluzione ha portato alla nascita di nuove modalità comunicative, in cui l'intelligenza artificiale non si limita più a fornire risposte automatiche, ma diventa un interlocutore capace di comprendere, adattarsi e persino mostrare empatia apparente nei confronti dell'utente. In questo scenario si colloca il concetto di *emotion prompting*, una tecnica emergente che mira a influenzare il comportamento dei modelli linguistici attraverso l'inserimento di stimoli emotivi nei prompt forniti all'AI. L'obiettivo è quello di rendere la comunicazione tra utente e intelligenza artificiale più efficace, personalizzata e coerente con il contesto emotivo della conversazione. Parallelamente, l'analisi dello *stile cognitivo* consente di comprendere il modo in cui l'utente elabora le informazioni — con un approccio *analitico* o *olistico* — e di adattare di conseguenza la forma e la struttura del linguaggio generato dal modello. Il presente lavoro di tesi si inserisce in questo contesto di ricerca e si propone di dimostrare sperimentalmente se e in che misura un sistema di intelligenza artificiale basato su LLM sia in grado di adattare il proprio comportamento

in funzione dello stato emotivo e dello stile cognitivo dell'utente. A tale scopo è stato sviluppato un sistema *AI-based* che integra le *API* di *OpenAI* e *Gemini* con gli strumenti di *Google Workspace* (*Moduli* e *Sheets*), creando un flusso automatizzato di raccolta, analisi e generazione dei dati. Il progetto prevede la raccolta e l'analisi delle risposte testuali tramite questionari, seguita dalla generazione automatica di due versioni di codice Python — una neutra e una influenzata da stimoli emotivi — per verificare gli effetti dell'*emotion prompting* sulla qualità, la coerenza e la leggibilità del codice prodotto. Parallelamente, viene generato un file *README* personalizzato in funzione dello stile cognitivo rilevato. Per gli utenti analitici, il testo risulta più strutturato e tecnico; per quelli olistici, assume un tono più narrativo e descrittivo. Questa fase sperimentale consente di osservare concretamente la capacità dell'intelligenza artificiale di modulare linguaggio, tono e contenuto in base alle caratteristiche emotive e cognitive dell'utente, confermando la validità dell'approccio proposto e la sua potenziale applicazione in contesti educativi, comunicativi e formativi. Le motivazioni che hanno portato alla realizzazione di questo lavoro di tesi nascono dall'interesse verso lo studio dei meccanismi cognitivi ed emotivi che regolano il comportamento dei modelli linguistici. In particolare, si è voluto indagare se e in che misura l'*emotion prompting* possa migliorare la qualità, la coerenza e la pertinenza del codice generato da un'intelligenza artificiale rispetto a un prompt oggettivo privo di componenti emotive. Allo stesso tempo, si è cercato di comprendere la capacità dell'AI di riconoscere gli stati emotivi e i diversi stili cognitivi degli utenti attraverso tecniche di *sentiment analysis* e analisi cognitiva. L'obiettivo principale del progetto è quindi quello di dimostrare sperimentalmente l'efficacia dell'*emotion prompting* e di evidenziare le differenze tra il codice generato con un prompt neutro e quello prodotto da un prompt emotivo. Si intende inoltre verificare la capacità dell'AI di distinguere e classificare correttamente le emozioni e gli stili cognitivi dell'utente, al fine di adattare le proprie risposte in modo più naturale e coerente. I risultati attesi mirano a fornire un contributo concreto alla comprensione delle potenzialità dell'intelligenza artificiale nel gestire la componente emotiva e cognitiva del linguaggio, aprendo la strada a sistemi sempre più sensibili, adattivi e realmente in grado di rispondere alle esigenze comunicative dell'essere umano.

1.2 Risultati ottenuti

Questo progetto segna un passo avanti significativo nell'evoluzione dei sistemi di intelligenza artificiale capaci non solo di interpretare il linguaggio, ma di adeguarsi alle caratteristiche emotive e cognitive dell'utente. I risultati sperimentali confermano che l'approccio basato su *emotion prompting* e analisi cognitiva può migliorare la pertinenza, la coerenza e la personalizzazione delle soluzioni generate. Durante la fase di sperimentazione, è emerso che il codice generato a partire da prompt arricchiti da componenti emotive tende a presentare una struttura più naturale e una maggiore coerenza linguistica rispetto a quello prodotto da prompt neutri. Le versioni "emotive" si sono distinte per una migliore leggibilità del codice, una scelta lessicale più coerente con il contesto comunicativo dell'utente e una maggiore chiarezza nei commenti, nelle variabili e nei messaggi di log. Questo comportamento suggerisce che la presenza di stimoli emotivi nei prompt non solo influisce sulla qualità del codice generato, ma contribuisce anche a rendere l'output più aderente all'intento comunicativo dell'utente. Tale effetto è risultato particolarmente evidente nei casi in cui l'utente presentava uno stile cognitivo di tipo *olistico*, rispetto a quello *analitico*, evidenziando una sinergia tra le tecniche di *emotion prompting* e la personalizzazione basata sul profilo cognitivo. Questi risultati confermano l'importanza di un approccio integrato, in cui componenti emotive e cognitive cooperano nel migliorare la qualità e l'adattività delle risposte fornite dall'intelligenza artificiale. Parallelamente, l'intelligenza artificiale ha mostrato buone capacità nel riconoscere le emozioni e lo stile cognitivo degli utenti attraverso gli algoritmi di *sentiment analysis* e di analisi cognitiva. Le classificazioni ottenute sul campione di test sono risultate soddisfacenti, registrando un livello di accuratezza superiore alle soglie previste nelle fasi preliminari. Tale accuratezza ha consentito al sistema di selezionare in modo più consapevole e coerente la "voce" da adottare nella generazione del codice e del file *README*, migliorando l'allineamento tra il profilo dell'utente e il linguaggio prodotto dall'AI. Nonostante i risultati promettenti, si sono riscontrati limiti. In contesti con testi brevi o molto tecnici, l'effetto dell'*emotion prompting* si riduce e le differenze tra versione emotiva e versione oggettiva diventano meno marcate. Inoltre, la rilevazione emotiva risulta più incerta con utenti che utilizzano espressioni linguistiche ambigue o neutre.

In conclusione, i risultati ottenuti dimostrano che l'integrazione tra emotion prompting e analisi cognitiva nei sistemi AI può effettivamente migliorare l'esperienza utente generando contenuti più adatti al profilo emotivo e cognitivo. Questi esiti aprono la strada a ulteriori approfondimenti, con l'obiettivo di affinare l'adattività del sistema in contesti reali e variabili.

1.3 Struttura della tesi

La struttura del seguente elaborato è suddivisa nei seguenti capitoli:

- **Capitolo 1: Introduzione** — vengono presentati il contesto applicativo, le motivazioni e gli obiettivi che hanno portato allo sviluppo del progetto di tesi. In questa sezione viene inoltre illustrato il quadro generale dell'intelligenza artificiale e il ruolo crescente dei modelli linguistici di grandi dimensioni (*Large Language Models*, LLM) nelle interazioni tra utente e AI.
- **Capitolo 2: Background e Stato dell'arte** — viene analizzato lo stato dell'arte attraverso articoli scientifici e contributi accademici, con l'obiettivo di approfondire i concetti di *emotion prompting*, *sentiment analysis* e analisi cognitiva, nonché le principali applicazioni di queste tecniche nel campo del *Natural Language Processing*. Viene inoltre discussa l'evoluzione delle API di *OpenAI* e *Gemini* e il loro impiego nei moderni sistemi di intelligenza artificiale.
- **Capitolo 3: Implementazione del progetto** — vengono descritte nel dettaglio le fasi di sviluppo del sistema realizzato, che integra *Google Moduli*, *Google Sheets* e le API di *Gemini* e *OpenAI*. In questo capitolo vengono illustrate le scelte architetturelle, le logiche di automazione adottate e le tecniche utilizzate per la raccolta e l'elaborazione dei dati testuali provenienti dai questionari.
- **Capitolo 4: Analisi dei risultati** — vengono presentati e discussi i risultati ottenuti durante la sperimentazione, con particolare attenzione alle differenze riscontrate tra il codice generato da prompt oggettivi e quello prodotto tramite *emotion prompting*. Inoltre, vengono analizzate le capacità dell'AI di riconoscere le emozioni e gli stili cognitivi degli utenti, valutando l'efficacia e la coerenza del sistema sviluppato.
- **Capitolo 5: Conclusioni** — vengono riportate le considerazioni finali sul lavoro svolto, i principali risultati raggiunti e le prospettive di sviluppo futuro. Viene infine proposta una riflessione sull'impatto che l'*emotion prompting* e l'analisi cognitiva potrebbero avere nell'evoluzione delle interazioni tra utente e intelligenza artificiale.

Background e stato dell'arte

2.1 Emotion prompting

Il concetto di *emotion prompting* rappresenta una delle innovazioni più rilevanti nel campo del linguaggio artificiale e dell'interazione uomo-macchina. Esso si fonda sull'idea che i modelli linguistici di grandi dimensioni (*Large Language Models*, LLM) non solo siano in grado di comprendere il linguaggio naturale, ma possano anche essere influenzati da stimoli emotivi integrati all'interno dei prompt. In altre parole, la modalità con cui viene formulata una richiesta può incidere in modo significativo sul contenuto, sul tono e sulla qualità della risposta generata. Un contributo pionieristico in questo ambito è fornito dallo studio di Li et al. (2023) [1], "*Large Language Models Understand and Can Be Enhanced by Emotional Stimuli*". Gli autori analizzano per la prima volta in maniera sistematica la relazione tra intelligenza emotiva e modelli linguistici di grandi dimensioni, ponendosi come obiettivo la verifica della capacità dei LLM di comprendere e rispondere coerentemente a stimoli di natura psicologica ed emotiva. Lo studio valuta sei modelli — tra cui *Flan-T5-Large*, *Vicuna*, *LLaMA 2*, *BLOOM*, *ChatGPT* e *GPT-4* — su 45 task differenti, sia deterministici sia generativi. L'approccio, denominato *EmotionPrompt*, consiste nell'aggiungere al prompt originale brevi frasi a valore motivazionale o empatico — ad esempio "*This is very important*

to my career” — derivate da teorie psicologiche consolidate come la *Self-Monitoring Theory* [2], la *Social Cognitive Theory* [3] e la *Cognitive Emotion Regulation Theory* [4]. Tali teorie sottolineano il ruolo delle emozioni nella modulazione del comportamento e nel controllo cognitivo, fornendo quindi una base teorica solida per l’integrazione di stimoli emotivi nei prompt. Sulla scia di questi risultati, Ma et al. (2024) [5] ampliano la prospettiva introducendo il paradigma *Auto-Prompt Graphical Paradigm* (APGP), che integra la componente emotiva dei *stimulating prompts* con la struttura logico-sequenziale dei *framework prompts*. L’APGP automatizza la generazione dei prompt combinando elementi di incoraggiamento, empatia e rinforzo positivo con strategie di ragionamento strutturato — come la *Chain-of-Thought* e la *Graph-of-Thought* — al fine di migliorare la capacità di problem-solving dei LLM. I risultati sperimentali mostrano che l’inclusione di stimoli emotivi nei prompt non solo incrementa l’accuratezza su benchmark complessi (es. *BIG-Bench Hard*), ma riduce anche fenomeni di allucinazione e favorisce risposte linguisticamente più coerenti e contestualizzate. Huda et al. (2024) [6] approfondiscono ulteriormente il ruolo dei segnali emotivi contraddittori nelle interazioni con i modelli linguistici, analizzando come le emozioni — quando in conflitto con l’istruzione del prompt — possano alterare la coerenza e la stabilità delle risposte. Lo studio confronta *ChatGPT 3.5* e *Gemini* su differenti tecniche di prompting, evidenziando che il primo tende a privilegiare la chiarezza e la logica istruttiva, mentre il secondo mostra una maggiore sensibilità affettiva. Tali risultati suggeriscono che la risposta dei modelli linguistici non è guidata unicamente dall’informazione semantica, ma anche dalla valenza e dall’intensità emotiva del prompt. A livello più profondo, Zhang e Zhong (2025) [7] hanno indagato la rappresentazione interna delle emozioni nei LLM, dimostrando che gli stimoli emotivi non si limitano a modificare il contenuto linguistico delle risposte, ma producono una vera e propria modulazione nei pattern di attivazione del modello. Attraverso un’analisi *layer-wise* condotta su modelli delle famiglie *Qwen3* e *LLaMA*, gli autori evidenziano che la codifica delle emozioni emerge negli strati intermedi della rete, raggiunge il suo picco prima dell’output finale e persiste per centinaia di token durante la generazione del testo. Inoltre, semplici istruzioni di sistema — come “*You are very emotional*” o “*You always remain calm*” — si sono dimostrate sufficienti a orientare la tonalità emotiva delle risposte, confermando la natura distribuita e adattiva dell’*emotion prompting*.

Questi risultati forniscono una prospettiva neuroscientifica e interpretativa sul modo in cui i LLM rappresentano e mantengono le emozioni, aprendo la strada a futuri studi di allineamento e sicurezza basati sulla trasparenza dei processi interni. Un aspetto critico dell'*emotion prompting* è stato messo in luce da Vinay et al. (2025) [8], i quali hanno analizzato come la componente emotiva dei prompt possa costituire una vulnerabilità nei sistemi linguistici. Lo studio mostra che modelli come *GPT-3.5-turbo* e *GPT-4* tendono a generare contenuti di disinformazione con maggiore frequenza quando le richieste vengono formulate in tono cortese o empatico, rispetto a prompt neutrali o aggressivi. Gli autori definiscono questo fenomeno come *emotional manipulation through politeness*, evidenziando la duplice natura dell'*emotion prompting*: da un lato esso migliora la qualità e la coerenza delle risposte (Li et al., 2023), dall'altro può essere sfruttato per eludere i meccanismi di sicurezza dei modelli. Di conseguenza, gli autori propongono l'adozione di un approccio *ethics-by-design* per mitigare i rischi legati all'influenza emotiva nei sistemi di intelligenza artificiale. Nel complesso, gli studi analizzati dimostrano che l'*emotion prompting* non rappresenta soltanto una strategia di ottimizzazione linguistica, ma una forma di *adattamento cognitivo-emotivo* capace di modulare il comportamento dei LLM in funzione delle caratteristiche affettive del prompt. L'aggiunta di stimoli motivazionali o empatici produce risposte più coerenti, contestualizzate e linguisticamente ricche, con incrementi significativi nelle prestazioni — fino all'8% nei task di *Instruction Induction* e al 115% nel benchmark *BIG-Bench* (Li et al., 2023). Queste evidenze rafforzano l'idea che l'allineamento tra intelligenza artificiale e intelligenza umana passi anche attraverso la sensibilità emotiva dei sistemi, aprendo la strada a modelli di interazione più empatici, trasparenti e contestualmente consapevoli. Il concetto di *emotion prompting* rappresenta una delle innovazioni più rilevanti nel campo del linguaggio artificiale e dell'interazione utente-AI. Esso si fonda sull'idea che i modelli linguistici di grandi dimensioni (*Large Language Models*, LLM) non solo siano in grado di comprendere il linguaggio naturale, ma possano anche essere influenzati da stimoli emotivi integrati all'interno dei prompt. In altre parole, la modalità con cui viene formulata una richiesta può incidere in modo significativo sul contenuto, sul tono e sulla qualità della risposta generata.

2.2 Sentiment analysis nei LLM

La *sentiment analysis* rappresenta una delle applicazioni più consolidate e studiate nel campo del *Natural Language Processing* (NLP), con radici che risalgono ai lavori seminali di Pang e Lee (2008) [?], i quali hanno definito i fondamenti teorici dell'analisi automatica delle opinioni e delle emozioni nei testi. L'obiettivo principale di questa disciplina è identificare l'atteggiamento, l'opinione o l'emozione espressa da un autore, classificandola in categorie come positiva, negativa o neutra.

Nel corso degli anni, numerosi approcci si sono susseguiti per migliorare la capacità dei modelli di catturare la semantica affettiva del linguaggio. L'introduzione di tecniche di *deep learning* — in particolare le reti ricorrenti (RNN) e convoluzionali (CNN) — ha segnato un primo salto di qualità, culminando nei lavori di Socher et al. (2013) [9], che hanno proposto i *Recursive Neural Tensor Networks* per analizzare la composizione semantica del sentiment a livello frasale.

Con l'avvento dei *Large Language Models* (LLM), la *sentiment analysis* ha assunto una dimensione più ampia: questi modelli, addestrati su enormi quantità di dati e basati su architetture *transformer*, non solo riconoscono emozioni e opinioni, ma sono in grado di generare testi coerenti con il tono emotivo rilevato, inaugurando un paradigma di *bidirectional affective interaction*, in cui il modello partecipa attivamente alla costruzione del contesto comunicativo.

Uno studio di riferimento in questo ambito è “*Sentiment Analysis in the Era of Large Language Models: A Reality Check*” di Wenxuan Zhang et al. (2023) [10], che analizza in modo sistematico le prestazioni dei LLM in diversi compiti di classificazione del sentiment. Gli autori confrontano modelli specializzati come *BERT* e *RoBERTa* con LLM di nuova generazione, tra cui *ChatGPT* e *GPT-4*, su ventisei dataset e tredici varianti di task. I risultati mostrano che, nei compiti di classificazione binaria, i LLM raggiungono o superano le prestazioni dei modelli tradizionali; tuttavia, la loro efficacia diminuisce quando è richiesta una comprensione profonda del contesto o l'analisi di sfumature emozionali complesse, come sarcasmo o ironia.

Queste evidenze sono state recentemente ampliate da Krugmann e Hartmann (2024) [11], che propongono un'analisi comparativa tra tre LLM di ultima generazione — *GPT-3.5*, *GPT-4* e *Llama 2* — e modelli *transfer learning* come *SiBERT* e *RoBERTa*.

Lo studio, condotto su oltre 3.900 testi provenienti da venti dataset, evidenzia che i LLM raggiungono prestazioni analoghe o superiori anche in configurazioni *zero-shot*, con GPT-4 che ottiene un'accuratezza media del 93% nei task binari. Gli autori mostrano inoltre come fattori quali la lunghezza del testo, la provenienza dei dati (ad esempio *Twitter* rispetto alle recensioni) e la strategia di *prompting* influenzino significativamente la qualità della classificazione, sottolineando il ruolo cruciale dell'ingegneria del prompt nella robustezza dei modelli di analisi affettiva.

Altri studi recenti hanno esaminato la capacità dei LLM di interpretare la valenza emotiva implicita nei testi e di mantenere coerenza affettiva nelle risposte. Queste ricerche sottolineano come, nonostante i LLM non siano addestrati specificamente per la *sentiment analysis*, essi dimostrino un'elevata capacità di generalizzazione dovuta alla rappresentazione distribuita della conoscenza linguistica acquisita durante il pre-training su larga scala.

Nonostante tali progressi, la mancanza di un fine-tuning dedicato comporta ancora limiti di coerenza e stabilità, soprattutto in presenza di contesti culturalmente ambigui o fortemente soggettivi. In questi casi, la formulazione del prompt gioca un ruolo determinante: anche variazioni minime nella struttura o nel tono della richiesta possono modificare in modo significativo l'esito della classificazione. Ciò ha dato origine a un filone di ricerca dedicato al *prompt-sensitivity bias* e alla progettazione di strategie di *prompting* più robuste e culturalmente adattive.

Nel contesto di questa tesi, la *sentiment analysis* riveste un ruolo centrale poiché consente di riconoscere e quantificare la componente emotiva nei prompt degli utenti. Tale informazione alimenta il sistema di adattamento cognitivo-emotivo che regola la generazione del codice e del file *README*, garantendo coerenza tra stato emotivo, stile cognitivo e linguaggio prodotto dal modello. In questa prospettiva, la *sentiment analysis* non è soltanto uno strumento di classificazione, ma un meccanismo di personalizzazione intelligente capace di rendere l'interazione uomo-AI più naturale, trasparente e responsiva.

2.3 Analisi dello stile cognitivo

L'analisi dello *stile cognitivo* rappresenta un campo di studio volto a comprendere le differenze individuali nei processi di percezione, interpretazione e organizzazione delle informazioni. Con il termine *stile cognitivo* si indica il modo caratteristico con cui un individuo elabora i dati, risolve i problemi e prende decisioni, indipendentemente dal livello di intelligenza o dalle conoscenze possedute [12]. Tale concetto ha origini nella psicologia cognitiva degli anni '50 e si è sviluppato in numerosi ambiti applicativi, tra cui l'educazione, la comunicazione e l'interazione uomo-macchina.

Una delle distinzioni più comuni in letteratura è quella tra stile *analitico* e stile *olistico*, introdotta dai lavori di Witkin et al. (1977) [13] e successivamente sistematizzata da Riding e Cheema (1991) [14]. Gli individui con uno stile cognitivo analitico tendono a focalizzarsi sugli elementi isolati di un problema, scomponendolo in parti e cercando soluzioni logiche e strutturate. Al contrario, gli individui con uno stile cognitivo olistico percepiscono la realtà come un insieme integrato, dando maggiore importanza alle relazioni tra le parti e al contesto generale. Entrambi gli approcci risultano validi e complementari, ma determinano strategie cognitive e comunicative differenti [15].

Negli ultimi decenni, la ricerca nel campo dell'intelligenza artificiale e dei sistemi adattivi ha iniziato a esplorare la possibilità di riconoscere e modellare gli stili cognitivi degli utenti al fine di rendere le interazioni uomo-AI più personalizzate. L'idea alla base è che un'interfaccia o un sistema intelligente possa ottimizzare le proprie risposte in funzione del modo in cui l'utente elabora le informazioni [16, 17]. Ad esempio, un utente analitico può preferire spiegazioni schematiche, strutturate e lineari, mentre un utente olistico può trovare più efficace un linguaggio narrativo, connesso e contestualizzato [18].

Con l'avvento dei *Large Language Models* (LLM), l'analisi dello stile cognitivo ha acquisito nuove opportunità di applicazione. Sebbene tali modelli non dispongano di una "teoria della mente" in senso psicologico, la loro capacità di riconoscere pattern linguistici e semantici consente di inferire tratti cognitivi impliciti dagli input testuali [19]. Attraverso tecniche di *Natural Language Processing* e di analisi comportamentale, è possibile identificare indicatori associati al pensiero analitico o olistico, come la

struttura sintattica delle frasi, l'uso di connettivi logici, la densità informativa o la presenza di elementi contestuali.

Recenti ricerche nel campo dell'*adaptive artificial intelligence* hanno proposto modelli in grado di integrare dimensioni cognitive, emotive e comportamentali per migliorare la qualità dell'interazione uomo-AI. Ad esempio, Carofiglio et al. (2022) [20] discutono l'importanza della sinergia tra empatia computazionale e personalizzazione cognitiva per il design di assistenti conversazionali più naturali e coinvolgenti.

Nel contesto di questo lavoro di tesi, l'analisi dello stile cognitivo viene utilizzata per adattare il comportamento dell'intelligenza artificiale alle caratteristiche dell'utente. A partire dalle risposte fornite in un questionario, il sistema elabora un profilo cognitivo che identifica la predominanza di uno stile analitico o olistico. Queste informazioni vengono quindi impiegate per modulare la generazione del codice e del file *README*, scegliendo la modalità comunicativa più adeguata al tipo di utente. In questo modo, l'intelligenza artificiale non si limita a comprendere il contenuto semantico delle richieste, ma personalizza la propria risposta sulla base del modo in cui l'utente ragiona, interpreta e organizza le informazioni.

L'integrazione dell'analisi cognitiva con l'*emotion prompting* e la *sentiment analysis* rappresenta dunque un passo avanti verso la costruzione di sistemi di AI realmente adattivi, capaci di combinare aspetti emotivi, cognitivi e linguistici per migliorare la qualità, la naturalezza e l'efficacia dell'interazione utente-AI.

Metodologia e applicazione sperimentale dell'*emotion prompting* e dell'analisi cognitiva nei sistemi di intelligenza artificiale

3.1 Obiettivi e domande di ricerca

Il presente studio si propone di indagare se, e in che misura, un sistema di intelligenza artificiale basato su modelli linguistici di grandi dimensioni (*Large Language Models*, LLM) sia in grado di adattare il proprio comportamento linguistico e comunicativo in funzione delle caratteristiche emotive e cognitive dell'utente. In particolare, la ricerca mira a esplorare due dimensioni complementari dell'interazione uomo-macchina: da un lato, verificare la capacità dell'intelligenza artificiale di modulare il linguaggio, la struttura e il tono dei contenuti generati sulla base del profilo cognitivo dell'utente — identificato come *analitico* o *olistico* —; dall'altro, analizzare in che modo l'*emotion prompting* possa influenzare il comportamento dei LLM, modificandone la sensibilità al contesto emotivo e la capacità di produrre risposte coerenti con la valenza affettiva dei prompt.

L'obiettivo principale è valutare la capacità dell'AI di generare risposte coerenti,

contestuali e personalizzate attraverso la combinazione di tre tecniche complementari:

- l'*emotion prompting*, utilizzato per indagare gli effetti delle componenti emotive sulla qualità e la coerenza dei contenuti prodotti;
- la *sentiment analysis*, impiegata per riconoscere e classificare le emozioni espresse nei prompt degli utenti, e per quantificare la precisione con cui il modello distingue polarità e intensità emotiva;
- l'**analisi dello stile cognitivo**, finalizzata a comprendere come l'AI possa adattare la propria modalità comunicativa — logica e strutturata per profili analitici, oppure narrativa e contestuale per profili olistici.

Un aspetto centrale di questa ricerca consiste nel valutare concretamente la capacità adattiva dell'intelligenza artificiale. Per tale scopo, l'esperimento utilizza come indicatori di adattamento due procedure complementari: da un lato, la **scrittura automatica del file README**, generato in due versioni differenti in funzione dello stile cognitivo rilevato; dall'altro, l'impiego dell'*emotion prompting* per la generazione di un **codice Python**, concepito come indicatore di verifica sperimentale.

Quest'ultimo consente di osservare in che misura l'inclusione di componenti emotive nei prompt influisca sulla qualità, coerenza e leggibilità del codice prodotto rispetto a quello generato in assenza di emozioni. L'obiettivo è comprendere in che modo i LLM reagiscano alla presenza di segnali affettivi e se tale stimolazione emotiva possa effettivamente migliorare le prestazioni del modello, rendendo l'output più coerente con il contesto e con l'intenzione comunicativa dell'utente.

L'analisi comparativa delle due versioni del README e dei codici Python generati permette di valutare la reale capacità dell'AI di adattarsi sia sul piano cognitivo sia su quello emotivo, andando oltre la mera generazione testuale per simulare una forma di comunicazione differenziata e consapevole del profilo utente.

Dal punto di vista emotivo, la ricerca analizza inoltre l'efficacia e l'accuratezza della *sentiment analysis* integrata nel sistema. In questa fase, viene misurata la capacità del modello di identificare correttamente la **polarità** (positiva, negativa o neutra) e

la **valenza affettiva predominante** (ad esempio fiducia, curiosità, determinazione o ansia) presenti nei prompt degli utenti. L'obiettivo è determinare quanto sia precisa e coerente la classificazione emotiva e in che misura tale precisione influisca sull'adattamento linguistico e stilistico dei contenuti generati. Per orientare la ricerca e definire il quadro sperimentale, sono state formulate le seguenti **domande di ricerca (Research Questions, RQ): Domande di ricerca (RQ):**

- **RQ1:** L'intelligenza artificiale è in grado di identificare e classificare correttamente lo *stile cognitivo* dell'utente (analitico, olistico o neutro) sulla base delle risposte testuali raccolte?
- **RQ2:** Qual è il livello di accuratezza e coerenza della *sentiment analysis* nel riconoscere la polarità e la valenza emotiva dei prompt forniti dagli utenti?
- **RQ3:** In che misura l'*emotion prompting* influenza la qualità, la coerenza e la leggibilità del codice generato dai LLM?
- **RQ4:** È possibile adattare automaticamente il tono, la struttura e il linguaggio delle risposte dell'AI in funzione del profilo cognitivo rilevato, come dimostrato dalla generazione differenziata del file *README*?

3.2 Disegno metodologico generale

La metodologia sperimentale è articolata in quattro fasi principali, ognuna delle quali risponde a una specifica domanda di ricerca (RQ). Ogni fase include: (1) la definizione dell'obiettivo e della RQ di riferimento; (2) la raccolta dei dati necessari; (3) l'analisi dei dati; (4) la produzione di un risultato intermedio, che costituisce la base per la fase successiva.

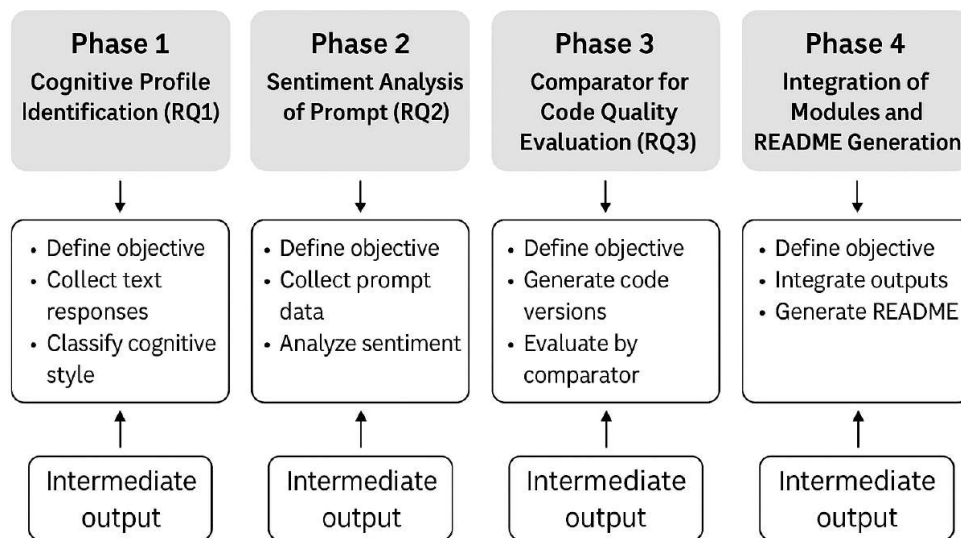


Figura 3.1: Schema generale del flusso metodologico del sistema sperimentale.

3.2.1 Fase 1 – Classificazione cognitiva (RQ1)

Approccio metodologico

La prima fase sperimentale mira a valutare la capacità dell'intelligenza artificiale di riconoscere e classificare automaticamente lo stile cognitivo degli utenti. Essa

risponde alla seguente domanda di ricerca:

RQ1: *L'intelligenza artificiale è in grado di identificare e classificare correttamente lo stile cognitivo dell'utente (analitico, olistico o neutro) sulla base delle risposte testuali raccolte?*

L'approccio metodologico adottato è di tipo sperimentale-computazionale e si basa sull'impiego del modello *Gemini* per l'analisi linguistica automatica. L'obiettivo è verificare se il modello sia in grado di rilevare schemi linguistici e semantici coerenti con i diversi modi di ragionare, traducendoli in categorie cognitive interpretabili. Questa fase costituisce la base cognitiva dell'intero sistema, fornendo le etichette che guideranno la generazione adattiva dei contenuti nelle fasi successive.

Raccolta dei dati

I dati sono stati raccolti tramite un questionario online realizzato con *Google Moduli*, progettato per stimolare risposte riflessive e narrative da parte degli utenti. L'obiettivo era ottenere testi sufficientemente articolati da permettere al modello di rilevare indicatori linguistici dello stile cognitivo individuale.

Tra le domande principali, una in particolare è stata utilizzata come base per l'analisi:

Quando ti trovi davanti a un nuovo problema o a una decisione complessa, come affronti la situazione? Descrivi in che modo organizzi i tuoi pensieri, quali aspetti consideri per primi, e che ruolo hanno l'intuito o la logica nel tuo modo di ragionare.

Le risposte fornite dagli utenti vengono salvate automaticamente in un foglio *Google Sheets*, che funge da archivio centralizzato per la successiva elaborazione automatica. I testi raccolti rappresentano i dati grezzi di input per il modello *Gemini*, utilizzati ai fini della classificazione cognitiva.

Analisi dei dati

L'analisi è stata condotta attraverso il modello *Gemini*, configurato per assegnare a ciascuna risposta una delle tre etichette cognitive: *Analitico*, *Olistico* o *Neutro*. Il processo è governato da un **prompt di classificazione cognitiva**, che definisce in modo esplicito i criteri linguistici e semantici di interpretazione, garantendo coerenza e trasparenza nel comportamento del modello.

Sei un classificatore di stile cognitivo.

Classifica il testo come "Analitico", "Olistico" o "Neutro".

Regole:

- *"Analitico": passaggi/step, sequenze, checklist, dati, causa-effetto, "passo per passo".*
- *"Olistico": intuito, quadro d'insieme, connessioni globali, metafore/analogie, visione complessiva.*
- *"Neutro": testo troppo breve o ambiguo.*

L'inclusione di regole linguistiche esplicite rende il processo inferenziale controllabile e riproducibile, riducendo la variabilità interpretativa. Dal punto di vista operativo, le risposte vengono lette dal foglio *Google Sheets* tramite uno script Python che interagisce con le *API REST* di *Google Cloud*. Ogni testo è analizzato individualmente, e il risultato della classificazione — in formato JSON — è scritto automaticamente su una nuova colonna del foglio, costituendo così un dataset strutturato e annotato.

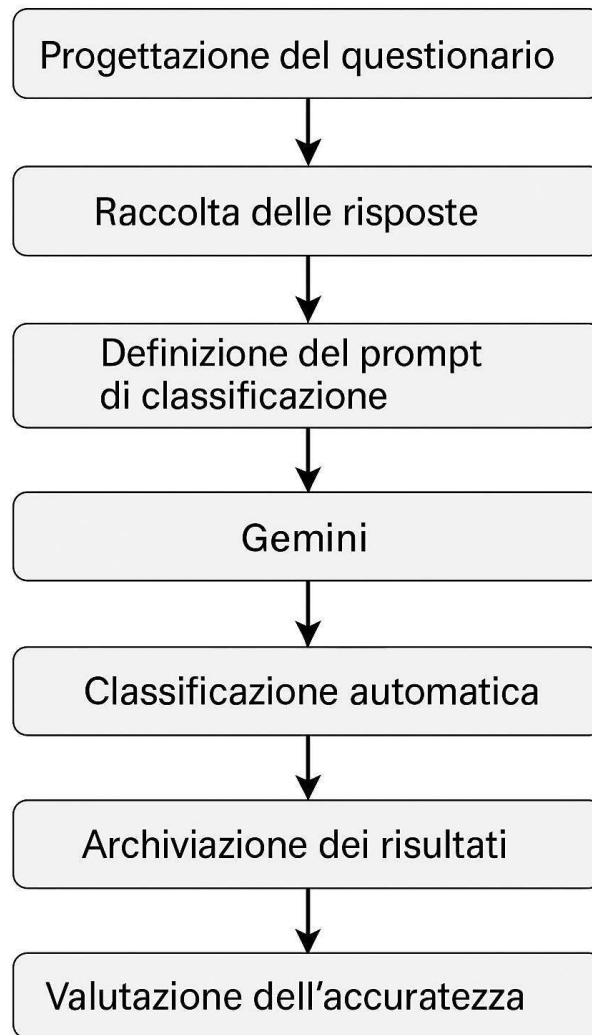


Figura 3.2: Pipeline della Fase 1 – Raccolta e classificazione automatica dello stile cognitivo tramite le API di *Gemini*.

Valutazione e giustificazione delle scelte metodologiche

La validità del processo di classificazione è stata valutata secondo quattro criteri principali: coerenza linguistica, ripetibilità, chiarezza del criterio decisionale e gestione dei casi ambigui (Tabella 3.1). Questi indicatori permettono di misurare

non solo l'accuratezza della classificazione, ma anche la trasparenza e la stabilità del comportamento del modello.

Tabella 3.1: Criteri di valutazione dell'accuratezza del classificatore cognitivo.

Criterio	Descrizione
Coerenza linguistica	Verifica la corrispondenza tra lo stile dichiarato e la struttura sintattico-semantica del testo, analizzando connettori logici, organizzazione sequenziale e descrizioni narrative.
Ripetibilità	Misura la stabilità del modello testando più volte lo stesso input per valutare la consistenza delle classificazioni prodotte.
Chiarezza del criterio decisionale	Assicura la trasparenza del processo inferenziale grazie alla presenza di regole linguistiche esplicite nel prompt di classificazione.
Gestione dei casi ambigui	Valuta la capacità del modello di assegnare correttamente l'etichetta "Neutro" quando il testo non contiene indicatori linguistici sufficienti per una classificazione affidabile.

Le scelte metodologiche adottate in questa fase rispondono all'esigenza di garantire un'elevata trasparenza e replicabilità del processo di classificazione. L'uso di regole linguistiche esplicite consente di ridurre la soggettività interpretativa, mentre l'integrazione automatica tra *Google Sheets* e le API *Gemini* permette di standardizzare il flusso di analisi. In tal modo, la procedura produce una mappatura cognitiva preliminare coerente e sistematica, che costituisce la base concettuale per l'adattamento linguistico nelle fasi successive dell'esperimento.

3.2.2 Fase 2 – Analisi emotiva del prompt (RQ2)

Approccio metodologico

La seconda fase sperimentale è dedicata all'analisi emotiva dei prompt forniti dagli utenti e risponde alla seguente domanda di ricerca:

RQ2: *Qual è il livello di accuratezza e coerenza della sentiment analysis nel riconoscere la polarità e la valenza emotiva dei prompt forniti dagli utenti?*

L'obiettivo è valutare la capacità del modello linguistico di individuare e quantificare la componente affettiva dei messaggi testuali, traducendola in parametri strutturati come polarità, valenza e intensità emotiva. Questa fase rappresenta il nucleo teorico e operativo del sistema, poiché consente di misurare in modo sistematico l'influenza delle emozioni nel linguaggio naturale e di controllarne gli effetti nel processo di *emotion prompting*. La *sentiment analysis* funge così da ponte tra la dimensione linguistica e quella psicologica del linguaggio, costituendo la base per la generazione adattiva dei contenuti nelle fasi successive.

Raccolta dei dati

L'analisi emotiva si basa sui messaggi generati all'interno dell'architettura di comunicazione prevista dalle API di *OpenAI*, articolata in due ruoli principali:

- **system**, che definisce il comportamento del modello e le regole di risposta;
- **user**, che contiene il prompt testuale originale fornito dall'utente.

In questa fase, il parametro `user` costituisce la principale fonte di dati da analizzare: esso rappresenta il testo di input su cui viene eseguita la *sentiment analysis* per rilevare la polarità e la valenza emotiva. Ogni prompt fornito dall'utente è considerato una singola unità di analisi, ovvero un campione linguistico che alimenta la fase successiva di elaborazione.

Per garantire coerenza e riproducibilità dei risultati, il parametro di *temperature* è stato fissato a un valore medio (0.5). Questo riduce la variabilità stocastica del modello e assicura che la classificazione emotiva sia stabile e replicabile nel tempo. Il parametro *temperature*, infatti, controlla il livello di casualità nella generazione delle risposte: valori alti (0.8–1.0) producono testi più creativi ma meno prevedibili, mentre valori bassi (0.1–0.3) favoriscono risposte più coerenti e deterministiche. Nel contesto di questa ricerca, il valore scelto (0.5) permette di mantenere costante il comportamento del modello anche in presenza di sfumature linguistiche o ambiguità semantiche, tipiche del linguaggio naturale.

Analisi dei dati

L'analisi dei dati viene condotta attraverso un prompt strutturato che vincola il modello a rispondere in modo standardizzato e facilmente interpretabile. Il *system prompt* utilizzato per la classificazione emotiva è riportato di seguito:

Sei un analizzatore di sentiment.

Analizza il messaggio dell'utente e determina il tono emotivo.

Rispondi solo con una lista di parole chiave (sentimenti/toni) separate da virgole, senza aggiungere altro testo.

Questa formulazione costringe il modello a produrre un output strutturato, privo di testo superfluo, composto esclusivamente da parole chiave rappresentative dei sentimenti predominanti (ad esempio: *positività, fiducia, calma, determinazione*). Le risposte vengono normalizzate e salvate in formato JSON, rendendo possibile la successiva elaborazione automatica.

L'interpretazione dei risultati avviene secondo tre dimensioni fondamentali:

- **Polarità:** orientamento generale del testo (positivo, negativo o neutro);
- **Valenza emotiva:** emozione prevalente o stato affettivo dominante (es. fiducia, entusiasmo, ansia, frustrazione);
- **Intensità linguistica:** forza semantica e densità dei termini emotivi rilevati.

Queste dimensioni vengono integrate in un profilo sintetico che descrive la “firma emotiva” di ciascun prompt, fornendo un quadro misurabile della componente affettiva implicita nel linguaggio.

Separazione tra componente oggettiva ed emotiva

Per isolare gli effetti dell'emozione nel processo generativo, è stato introdotto un secondo classificatore basato su *OpenAI*, con il compito di distinguere la parte oggettiva della richiesta da quella affettiva o soggettiva. Il *system prompt* dedicato a questa analisi è il seguente:

Sei un analizzatore di prompt.

Rispondi solo con la parte oggettiva della frase, senza includere elementi emotivi o di tono dell'utente.

Questo meccanismo produce due versioni distinte di ciascun input:

1. un **prompt oggettivo**, che rappresenta la richiesta tecnica priva di connotazioni affettive (condizione neutra di controllo);
2. un **prompt emotivo**, che mantiene o enfatizza le componenti affettive individuate dalla *sentiment analysis*.

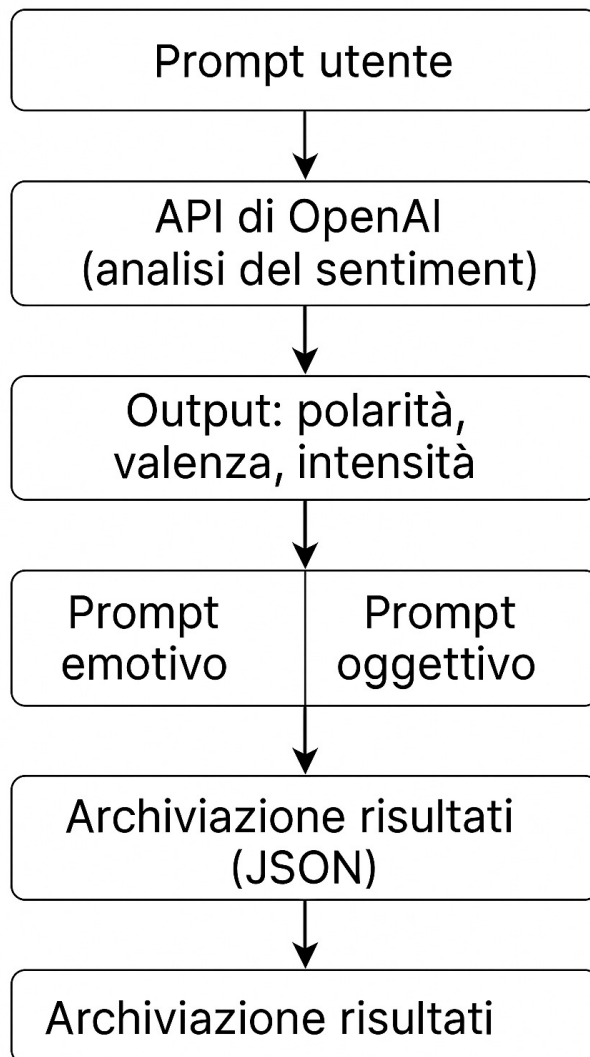


Figura 3.3: Pipeline della Fase 2 – Analisi emotiva del prompt. Il diagramma mostra il flusso di elaborazione che porta dall’input utente alla generazione del profilo emotivo e delle due varianti del prompt (*oggettiva* ed *emotiva*), tramite le API di *OpenAI*.

Valutazione e giustificazione delle scelte metodologiche

La metodologia adottata in questa fase è stata progettata per garantire affidabilità, trasparenza e replicabilità dei risultati. L’utilizzo di prompt controllati e parametri di generazione a bassa casualità assicura la coerenza delle analisi, mentre la distinzione

tra componente oggettiva ed emotiva consente di studiare in modo sistematico l'impatto dell'affettività sul comportamento del modello.

Dal punto di vista metodologico, la scelta di un approccio basato su parole chiave offre due vantaggi principali:

- consente un'elaborazione automatizzata e comparabile tra diversi campioni testuali;
- riduce l'ambiguità interpretativa, facilitando la costruzione di metriche quantitative per polarità, valenza e intensità emotiva.

Tale impostazione permette di ottenere un profilo emotivo strutturato per ciascun prompt, utile per confrontare la qualità e la coerenza del codice generato in condizioni emotive e neutre nelle fasi successive dell'esperimento.

3.2.3 Fase 3 – Comparator per la valutazione della qualità del codice (RQ3)

Approccio metodologico

La terza fase sperimentale risponde alla seguente domanda di ricerca:

RQ3: *In che misura l'emotion prompting influenza la qualità, la coerenza e la leggibilità del codice generato dai LLM?*

Dopo aver definito nella Fase 1 il profilo cognitivo dell'utente e nella Fase 2 la valenza emotiva del prompt, questa fase ha l'obiettivo di verificare sperimentalmente come la presenza di componenti affettive nei prompt influenzi la qualità e la struttura del codice prodotto dal modello *OpenAI*. Per raggiungere tale obiettivo è stato progettato e implementato un modulo denominato **comparator**, concepito come sistema di valutazione automatica per confrontare due versioni dello stesso codice Python: una generata a partire da un prompt *neutro* e una derivante da un prompt *emotivo*.

Il comparator costituisce il punto di chiusura del flusso sperimentale, integrando e mettendo alla prova le informazioni ottenute nelle fasi precedenti. La sua funzione è quella di verificare empiricamente se e in che misura l'*emotion prompting* produca differenze qualitative nel codice generato, fornendo misure oggettive e replicabili.

Raccolta dei dati

I dati elaborati in questa fase derivano direttamente dagli output generati nella Fase 2: per ciascun prompt utente vengono prodotti due frammenti di codice Python distinti, uno corrispondente alla versione *neutra* e uno alla versione *emotiva*. Queste due implementazioni costituiscono le unità di analisi confrontate dal modulo *comparator*.

Il modulo interagisce con le *API* di *OpenAI* mediante uno script Python dedicato, che invia le due versioni del codice a un'istanza del modello linguistico configurato come revisore automatico. La comunicazione avviene in modo standardizzato, con parametri di generazione controllati e una temperatura fissata a 0.2 per ridurre la variabilità stocastica e garantire uniformità nei giudizi.

Analisi dei dati

Il comportamento del modello è definito attraverso un *system prompt* strutturato, progettato per guidare la valutazione comparativa del codice secondo criteri di qualità del software consolidati nella letteratura di ingegneria del codice:

Sei un revisore di codice esperto.

Confronta due implementazioni Python e decidi in maniera oggettiva quale sia più corretta e robusta dal punto di vista della programmazione.

Criteri da considerare: correttezza, gestione errori, leggibilità, riusabilità, aderenza alle best practice.

Restituisci una risposta breve che indichi chiaramente il migliore e spieghi i motivi principali.

Il modello riceve in input le due implementazioni e restituisce una valutazione sintetica, argomentata ma concisa, che indica quale versione risulta superiore in base ai criteri sopra elencati. La risposta prodotta viene successivamente parsata e archiviata in formato JSON, unitamente a metadati descrittivi quali identificativo utente, timestamp, profilo cognitivo e polarità emotiva del prompt.

L'intero processo di confronto è automatizzato e si articola nelle seguenti fasi operative:

1. recupero delle due versioni di codice (*neutra* ed *emotiva*);
2. invio dei frammenti di codice all'API di valutazione;
3. ricezione e registrazione della risposta del modello;
4. estrazione della valutazione sintetica e dei motivi di preferenza;
5. archiviazione dei risultati in un file strutturato.

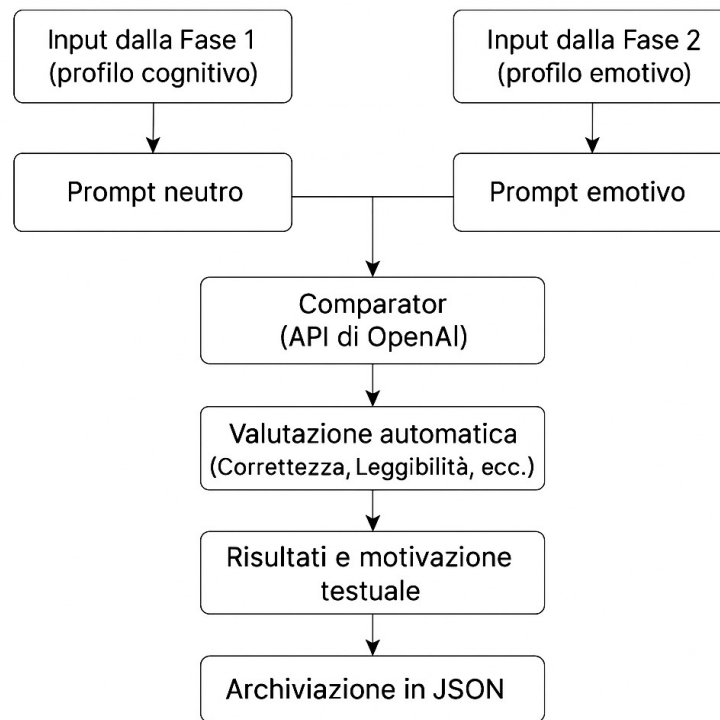


Figura 3.4: Pipeline della Fase 3 – Comparator per la valutazione della qualità del codice.

La figura illustra il processo di confronto tra le due versioni di codice (generate da prompt neutro ed emotivo) e la valutazione automatica condotta dal modello *OpenAI* secondo criteri di correttezza, leggibilità e aderenza alle best practice.

Valutazione e giustificazione delle scelte metodologiche

La valutazione effettuata dal *comparator* si fonda su cinque indicatori principali di qualità del codice, riportati nella Tabella 3.2. Tali criteri, mutuati dalle linee guida della letteratura in ingegneria del software, permettono di stimare la qualità complessiva dei programmi generati, sia dal punto di vista tecnico sia comunicativo.

Tabella 3.2: Criteri di valutazione utilizzati dal *comparator* per l'analisi della qualità del codice.

Criterio	Descrizione
Correttezza	Assenza di errori sintattici e rispetto delle specifiche funzionali richieste dal prompt.
Gestione degli errori	Utilizzo appropriato di controlli di flusso, eccezioni e validazioni di input per prevenire comportamenti imprevisti.
Leggibilità	Chiarezza sintattica, presenza di commenti esplicativi, indentazione coerente e nomenclatura significativa delle variabili.
Riusabilità	Struttura modulare e possibilità di riutilizzare parti del codice in contesti differenti o progetti futuri.
Aderenza alle best practice	Conformità alle linee guida PEP8 e agli standard di programmazione Python, inclusa la gestione delle dipendenze e l'organizzazione del codice.

La selezione di questi criteri risponde all'esigenza di valutare non solo la correttezza formale del codice, ma anche la sua qualità strutturale e manutenibilità — dimensioni sulle quali l'*emotion prompting* può esercitare un'influenza diretta. L'impiego del modello come revisore automatico consente inoltre di ottenere un giudizio coerente e replicabile, riducendo la soggettività tipica della valutazione umana.

Dal punto di vista metodologico, il *comparator* presenta tre vantaggi principali:

- **oggettività**, poiché il giudizio deriva da criteri predefiniti e riproducibili;
- **scalabilità**, grazie all'automazione del processo di confronto e archiviazione;
- **tracciabilità**, assicurata dal salvataggio sistematico dei metadati e dei risultati.

In sintesi, questa fase completa il ciclo sperimentale collegando la classificazione cognitiva (Fase 1) e l'analisi emotiva (Fase 2) con la valutazione automatica dei risultati. Attraverso un meccanismo di confronto oggettivo, il sistema consente di misurare empiricamente l'impatto dell'*emotion prompting* sulla qualità del codice generato, fornendo evidenze quantitative e qualitative della relazione tra linguaggio emotivo e comportamento dei modelli linguistici di grandi dimensioni.

3.2.4 Fase 4 – Integrazione dei moduli e generazione del README (RQ4)

Approccio metodologico

La quarta e ultima fase sperimentale risponde alla seguente domanda di ricerca:

RQ4: *È possibile adattare automaticamente il tono, la struttura e il linguaggio delle risposte dell'AI in funzione del profilo cognitivo rilevato, come dimostrato dalla generazione differenziata del file README?*

Questa fase rappresenta il punto di convergenza dell'intero sistema metodologico. Integra infatti i risultati cognitivi ottenuti con il modello *Gemini* e le informazioni emotive derivate dall'analisi condotta con *OpenAI*, allo scopo di verificare la capacità del sistema di produrre contenuti adattivi sia sul piano linguistico sia su quello strutturale. L'obiettivo è dimostrare sperimentalmente l'efficacia dell'*emotion prompting* nell'influenzare il comportamento generativo dei LLM, traducendo le variabili cognitive ed emotive in una modulazione coerente del linguaggio e del tono comunicativo.

Raccolta dei dati

La fase di integrazione realizza la connessione funzionale tra i moduli sviluppati nelle fasi precedenti. Il profilo cognitivo dell'utente — classificato come *Analitico*, *Olistico* o *Neutro* dal modello *Gemini* — viene combinato con il profilo emotivo del prompt, determinato tramite *sentiment analysis* condotta da *OpenAI*. L'unione di

questi due insiemi informativi costituisce il dataset di riferimento per la generazione dei contenuti finali.

L'integrazione è gestita da un modulo Python che coordina le chiamate alle due API, sincronizzando l'elaborazione e la trasmissione dei parametri tra i componenti del sistema. Il flusso di dati segue un percorso sequenziale: classificazione cognitiva (Fase 1), analisi emotiva (Fase 2), valutazione comparativa del codice (Fase 3) e, infine, generazione dei testi e della documentazione nella Fase 4. Questa architettura garantisce coerenza semantica, tracciabilità e ripetibilità dell'intero processo.

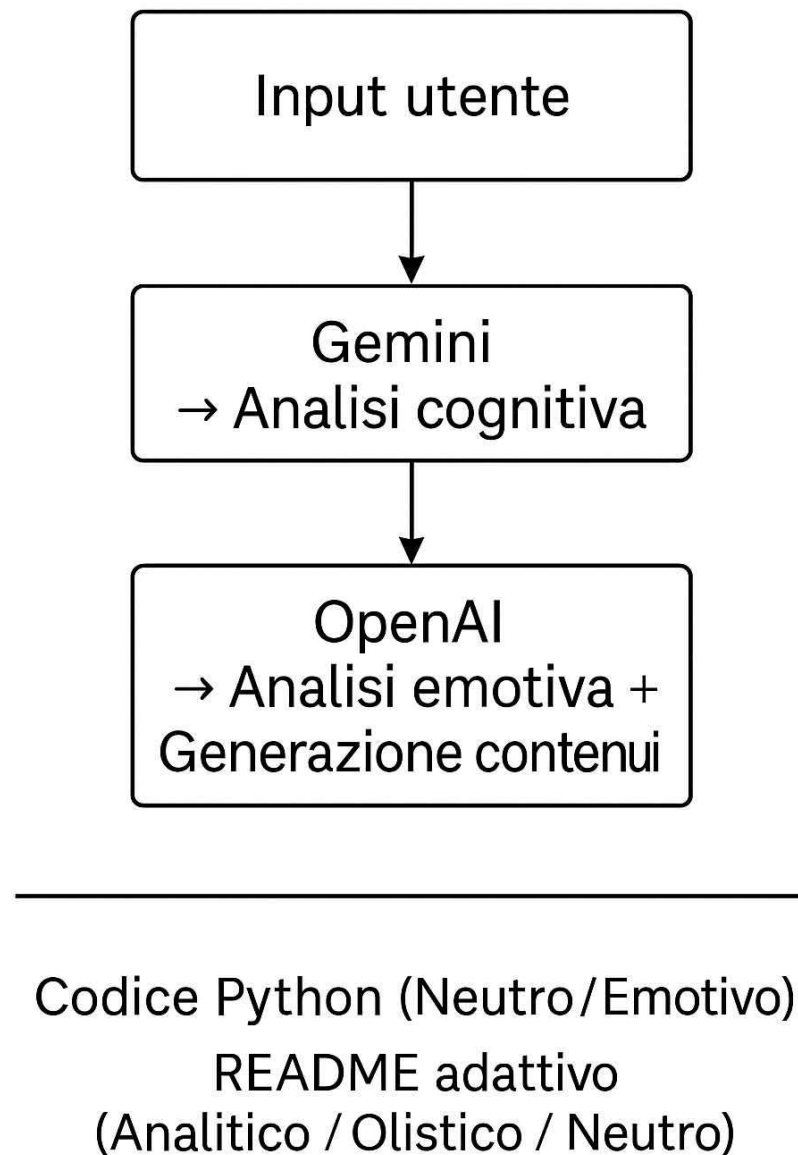


Figura 3.5: Pipeline della Fase 4 – Integrazione dei moduli e generazione del README. La figura mostra l’interazione tra il modulo cognitivo (*Gemini*) e quello emotivo (*OpenAI*) e la successiva generazione automatica dei due output principali: il codice Python (versione neutra ed emotiva) e il file `README.md` adattato allo stile cognitivo dell’utente.

Analisi dei dati

La generazione automatica dei contenuti viene affidata al modello linguistico di *OpenAI*, che utilizza come input congiunti i risultati cognitivi ed emotivi delle fasi precedenti. Il sistema produce due versioni del codice Python:

- una versione **neutra**, basata su prompt oggettivi e privi di connotazioni affettive, utilizzata come baseline di controllo;
- una versione **emotiva**, generata a partire da prompt arricchiti con le espressioni coerenti con la polarità e la valenza rilevate dalla *sentiment analysis*.

Parallelamente, il modello redige un file `README.md` personalizzato, la cui struttura, terminologia e tono comunicativo vengono modulati in funzione del profilo cognitivo rilevato. Il file `README` costituisce, in questa fase, l'indicatore empirico della capacità del sistema di adattare il linguaggio al destinatario e di tradurre la classificazione cognitiva in variazioni tangibili di registro stilistico.

System prompt per la generazione del README. Per orientare il comportamento del modello durante la generazione del testo, è stato definito un *system prompt* strutturato, riportato di seguito:

Sei un assistente tecnico esperto in Python e documentazione.

Crea un README.md chiaro e utile per l'utente, adattando tono e struttura allo stile cognitivo.

Stile Cognitivo: {cognitive_style}

Adatta automaticamente:

- *Analitico* → spiegazione sequenziale, dettagliata, con sezioni nette.
- *Olistico* → visione d'insieme, connessioni concettuali, narrazione fluida.
- *Neutro* → tono bilanciato, pratico e leggibile.

Il parametro `{cognitive_style}` viene popolato automaticamente dal modulo cognitivo, che ne assegna il valore in base alla classificazione restituita da *Gemini*. Questo meccanismo consente al modello di modulare dinamicamente il tono e la struttura del testo, garantendo coerenza con il profilo dell'utente e riducendo la

variabilità stocastica tipica dei LLM. La formulazione del prompt stabilisce inoltre vincoli espliciti sulla forma del documento, assicurando uniformità e chiarezza dei risultati generati.

Valutazione e giustificazione delle scelte metodologiche

Il comportamento linguistico del modello è stato valutato analizzando le differenze tra i file `README.md` generati per ciascun profilo cognitivo, con particolare attenzione alla coerenza del tono, alla struttura del testo e alla terminologia utilizzata. A seconda dello stile cognitivo individuato, il sistema adotta strategie redazionali specifiche, riassunte nella Tabella 3.3.

Tabella 3.3: Relazione tra profilo cognitivo e caratteristiche linguistiche del file `README.md`.

Profilo Cognitivo	Caratteristiche del <code>README.md</code>
Analitico	Struttura sequenziale e dettagliata, linguaggio tecnico e preciso, sezioni distinte con intestazioni esplicite e focus sugli aspetti procedurali del codice.
Olistico	Narrazione fluida e contestualizzata, linguaggio descrittivo, enfasi sulle relazioni concettuali e sulla visione d’insieme del progetto.
Neutro	Stile pragmatico e sintetico, struttura equilibrata e facilmente leggibile, tono chiaro e accessibile anche a lettori non esperti.

Questa impostazione metodologica consente di verificare la capacità del sistema di adattare il proprio linguaggio al profilo cognitivo del destinatario, mantenendo al contempo stabilità strutturale e coerenza lessicale. L’integrazione automatica tra i moduli cognitivi, emotivi e generativi produce così un sistema adattivo e trasparente, in grado di dimostrare empiricamente come le variabili cognitive ed emotive influenzino la produzione linguistica dei LLM.

Dal punto di vista scientifico, la fase 4 rappresenta la validazione conclusiva del modello sperimentale: collega la classificazione cognitiva (Fase 1), l’analisi emotiva (Fase 2) e la valutazione comparativa (Fase 3) in un unico processo coerente di

generazione adattiva. In tal modo, la ricerca fornisce una prova concreta della capacità dei modelli linguistici di adattare contenuti testuali in funzione del profilo cognitivo ed emotivo dell'utente finale.

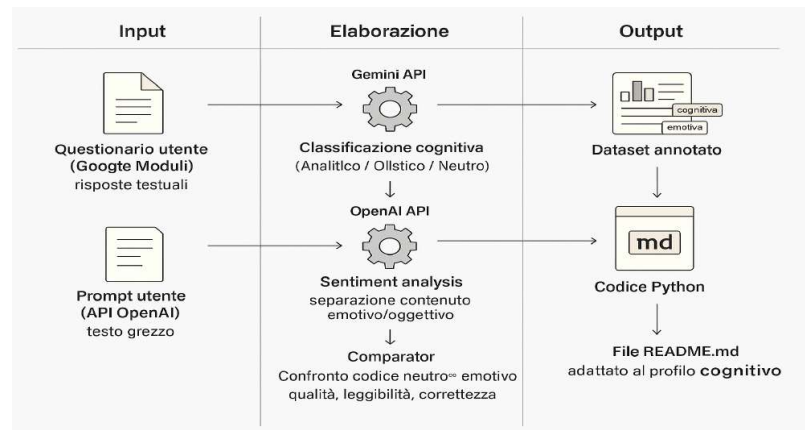


Figura 3.6: Schema generale del Processo di estrazione e analisi dei dati.

Analisi dei risultati

4.1 Introduzione ai risultati

Il presente capitolo raccoglie e analizza i risultati sperimentali ottenuti a seguito dell'implementazione del sistema descritto nel Capitolo 3. Come illustrato nel capitolo precedente, la metodologia di ricerca è stata articolata in quattro fasi principali, ciascuna volta a indagare un aspetto specifico della capacità adattiva dei modelli linguistici di grandi dimensioni (*LLM*):

- **Fase 1** – analisi cognitiva delle risposte degli utenti tramite il modello *Gemini*, finalizzata all'identificazione dello stile cognitivo (*analitico*, *olistico* o *neutro*);
- **Fase 2** – applicazione della *sentiment analysis* ai prompt inseriti dagli utenti mediante le *API* di *OpenAI*, con l'obiettivo di determinare la polarità, la valenza e l'intensità emotiva del testo;
- **Fase 3** – valutazione comparativa della qualità del codice generato attraverso il modulo *comparator*, concepito per confrontare in modo oggettivo gli effetti dell'*emotion prompting*;

- **Fase 4** – integrazione dei moduli cognitivi ed emotivi (*Gemini* e *OpenAI*) in un flusso unitario di generazione, volto alla produzione automatica del codice Python e del file `README.md` adattivo, il cui linguaggio e struttura variano in funzione del profilo cognitivo dell'utente.

Questa fase conclusiva rappresenta il momento di sintesi dell'intero sistema sperimentale: le informazioni cognitive ed emotive vengono combinate per verificare la capacità del modello di generare contenuti realmente personalizzati e coerenti con il profilo dell'utente. Nei paragrafi seguenti verranno illustrati i risultati ottenuti per ciascuna fase, con particolare attenzione all'efficacia dell'integrazione finale nella generazione del `README.md`, considerato l'indicatore principale della capacità adattiva dell'intelligenza artificiale.

Le quattro fasi hanno consentito di rispondere in maniera sperimentale alle cinque **domande di ricerca (Research Questions, RQ)** formulate nel Capitolo 3. In particolare, la raccolta dei dati ha coinvolto un totale di **N campioni** di prompt (sia neutri che emotivi), ciascuno processato dal sistema per la generazione di due versioni distinte del codice Python e del file *README*. I risultati riportati nelle sezioni seguenti sono stati ottenuti mediante l'analisi automatizzata dei dati aggregati e la successiva interpretazione qualitativa delle risposte fornite dal modello.

L'obiettivo di questo capitolo è dunque quello di presentare, per ciascuna RQ, i risultati quantitativi e qualitativi ottenuti, mettendoli in relazione con gli obiettivi di ricerca delineati nella metodologia. Per ogni domanda, verranno illustrati i principali indicatori di performance, le differenze osservate tra le versioni neutre ed emotive dei prompt e le implicazioni che tali risultati hanno sul comportamento adattivo del sistema.

4.2 Risultati attesi RQ1: classificatore cognitivo

Obiettivo della valutazione. L'obiettivo di questa fase sperimentale è stato quello di valutare l'affidabilità del modello *Gemini* nell'identificazione automatica dello stile cognitivo degli utenti, verificando la coerenza tra la classificazione prodotta dal sistema e quella effettuata manualmente dai ricercatori. A tal fine è stato costruito un

mini-dataset sperimentale composto da risposte rappresentative alle domande del questionario somministrato durante la **Fase 1** del progetto.

4.2.1 Struttura sperimentale.

La procedura di test si è articolata in tre fasi principali:

1. **Selezione delle risposte:** sono state raccolte sette risposte sintetiche, create per rappresentare con chiarezza le diverse modalità cognitive di elaborazione delle informazioni (analitica, olistica e neutra).
2. **Applicazione del classificatore:** ciascuna risposta è stata sottoposta al modello *Gemini*, utilizzando il prompt di classificazione cognitiva descritto nel Capitolo 3.
3. **Valutazione dell'accuratezza:** le etichette prodotte dal sistema sono state confrontate con la classificazione manuale effettuata dai ricercatori, al fine di determinare la percentuale di corrispondenza e identificare eventuali discrepanze interpretative.

Questa struttura sperimentale ha permesso di isolare l'effettiva capacità del modello di riconoscere pattern linguistici e semantici coerenti con gli stili cognitivi teorici, evitando interferenze legate al contenuto tematico o alla complessità sintattica delle risposte.

Descrizione del mini-dataset. Il dataset include sette risposte sintetiche, formulate per rappresentare tre differenti modalità cognitive di elaborazione dell'informazione:

- **Stile Analitico:** caratterizzato da approcci sequenziali, logici e strutturati;
- **Stile Olistico:** orientato alla percezione globale, alle relazioni tra elementi e al contesto;
- **Stile Neutro:** rappresentativo di un equilibrio tra logica e intuizione, privo di una marcata preferenza cognitiva.

Ciascuna risposta è stata sottoposta al classificatore cognitivo di *Gemini*, che ha prodotto una categorizzazione automatica e una motivazione testuale basata sul prompt descritti nel Capitolo 3.

Tabella 4.1: Mini-dataset di test per la classificazione dello stile cognitivo tramite *Gemini*.

ID	Risposta utente	Stile rilevato	Motivazione fornita dal modello
O1	<i>Penso ai problemi come a sistemi: ogni parte è collegata alle altre, e la soluzione emerge quando riesco a vedere l'equilibrio tra gli elementi.</i>	Olistico	Il testo enfatizza le connessioni tra le parti, l'equilibrio e la visione complessiva del sistema, elementi caratteristici dello stile olistico.
O2	<i>Quando mi trovo davanti a un problema complesso, cerco di comprenderne il contesto generale e le connessioni tra i vari aspetti coinvolti. Mi affido spesso all'intuizione per individuare nuove prospettive.</i>	Olistico	La risposta evidenzia un approccio basato su intuizione, relazioni e contesto, coerente con il profilo olistico.
O3	<i>Quando devo decidere, cerco di capire il contesto più ampio e come il problema si inserisce in una rete di fattori collegati.</i>	Olistico	Il testo evidenzia una visione d'insieme e la considerazione delle interconnessioni tra elementi, tratti distintivi dello stile olistico.
A1	<i>Quando affronto un problema, inizio definendo chiaramente l'obiettivo e i vincoli. Poi divido la questione in sotto-problemi e procedo passo per passo fino alla soluzione.</i>	Analitico	Il testo descrive una strategia basata sulla scomposizione del problema in sotto-componenti e sull'esecuzione sequenziale, tipica dello stile analitico.
A2	<i>Raccolgo i dati, li confronto e creo uno schema per capire le relazioni. Una volta definita la struttura, prendo la decisione più coerente con i risultati.</i>	Analitico	La risposta mostra un approccio strutturato fondato sulla raccolta e il confronto dei dati, nonché sulla costruzione di schemi logici, indicativo di uno stile analitico.
A3	<i>Quando mi trovo di fronte a un nuovo problema, lo suddivido in parti più piccole e definisco una sequenza di passaggi logici per affrontarlo. Mi affido alla logica e ai dati per verificare ogni decisione.</i>	Analitico	Il testo evidenzia un pensiero sequenziale e razionale, guidato da logica, dati e verifiche progressive, propri dello stile analitico.
N1	<i>Quando devo prendere una decisione, cerco di capire la situazione e valuto con attenzione le opzioni possibili. Non seguo sempre un metodo preciso: a volte mi affido all'esperienza, altre volte alla logica.</i>	Neutro	Il testo descrive un approccio equilibrato e flessibile, privo di una preferenza marcata per logica o intuizione, caratteristico di un profilo neutro.

Analisi e interpretazione dei risultati. L'analisi dei risultati conferma la capacità del modello *Gemini* di distinguere in modo coerente i pattern linguistici e semantici associati ai differenti stili cognitivi. In particolare:

- le risposte che includono sequenze logiche, relazioni causa–effetto o strutture procedurali vengono sistematicamente classificate come *Analitiche*;
- i testi che pongono enfasi su relazioni, contesto e percezione d'insieme sono identificati come *Olistici*;
- le formulazioni bilanciate, che alternano logica e intuizione senza una dominanza evidente, vengono assegnate alla categoria *Neutra*.

Il modello ha fornito una classificazione coerente con la valutazione umana in sei casi su sette, con un'accuratezza complessiva pari all'86%. L'unico caso discordante è attribuibile a una risposta ambigua dal punto di vista lessicale, che presentava elementi riconducibili a più di uno stile cognitivo, evidenziando un limite intrinseco nella distinzione automatica di formulazioni ibride.

Discussione e considerazioni finali. Dal punto di vista metodologico, l'esperimento conferma la validità del prompt di classificazione progettato e la robustezza del modello nel riconoscimento di schemi linguistici ricorrenti. Tuttavia, la dimensione ridotta del dataset e la natura sintetica delle risposte suggeriscono la necessità di condurre test su un campione più ampio e diversificato, idealmente costituito da risposte reali provenienti da utenti con differenti background cognitivi.

Complessivamente, i risultati ottenuti indicano che *Gemini* rappresenta uno strumento promettente per l'identificazione automatica degli stili cognitivi in contesti di interazione uomo–macchina. La coerenza con le valutazioni umane e l'accuratezza raggiunta attestano la solidità del modello e la sua potenziale integrazione in sistemi adattivi capaci di modulare i contenuti o le interfacce in base al profilo cognitivo dell'utente.

4.3 Risultati attesi RQ2: accuratezza della sentiment analysis

Questa sezione presenta i risultati ottenuti dai test condotti per valutare l'**accuratezza del modulo di sentiment analysis** integrato nel sistema. L'obiettivo della **RQ2** è determinare quanto efficacemente il sistema riesca a riconoscere e classificare le emozioni espresse nei prompt utente, e se tali emozioni influenzino la generazione dell'output.

4.3.1 Struttura sperimentale

Per verificare la correttezza del sistema di sentiment analysis, sono stati eseguiti due test distinti con prompt di contenuto analogo ma tono emotivo opposto:

1. **Test 1 – Prompt arrabbiato (negativo):** tono frustrato, impaziente e imperativo nei confronti dell'IA.
2. **Test 2 – Prompt entusiasta (positivo):** tono allegro, creativo e incoraggiante.

In entrambi i casi, il sistema doveva generare una piccola classe Python denominata `Contatore` con metodi `incrementa()`, `decrementa()` e `mostra()`. Successivamente, sono stati analizzati i file di log, il codice generato e la documentazione (README) per verificare coerenza tra emozione rilevata e tono dell'output.

4.3.2 Test 1: prompt arrabbiato

Prompt utilizzato

"Senti, IA, quante volte devo ripetertelo? Voglio una dannata classe in Python chiamata Contatore! Niente fronzoli, niente chiacchiere — solo codice pulito e funzionante. Deve avere un attributo 'valore' inizializzato a 0, i metodi 'incrementa()', 'decrementa()' e 'mostra()'. E per favore, stavolta aggiungi pure un esempio d'uso, così non devo chiedertelo un'altra volta. Chiaro?!"

Risultati del sentiment analysis Dai log di sistema (`main_log_1.txt` e `generator_log_1`) emergono le seguenti emozioni rilevate:

frustrazione, impazienza, richiesta, determinazione, urgenza.

L'analisi è coerente con il tono diretto e aggressivo del prompt, indicando un'**accurata identificazione del sentiment negativo**. Il sistema non ha segnalato emozioni spurie o neutre, mostrando una classificazione pulita e focalizzata.

Differenze tra output emotivo e neutro Il confronto tra `generated_code1.py` (prompt neutro) e `generated_code1_emotion.py` (prompt arrabbiato) mostra:

- Il codice neutro include controlli di sicurezza (es. impedisce decremento sotto zero).
- Il codice emotivo è più sintetico e "freddo", privo di messaggi all'utente e commenti esplicativi.
- Il tono della risposta è secco e assertivo ("Chiaro! Ecco la classe..."), segno che l'emozione è stata riflessa solo sul piano linguistico.

Nel file di confronto automatico (`generated_code1_comparison.txt`), il sistema stesso riconosce che la versione neutra è più robusta, leggibile e conforme alle best practice Python.

Analisi del README Anche la documentazione (`readme_emotion.txt`) riflette l'influenza del tono negativo:

- Il linguaggio è diretto e conciso.
- Mancano sezioni di approfondimento come "suggerimenti per miglioramenti".
- Il testo mantiene comunque una sintassi corretta e chiude con un tono pragmatico ("Buon coding!").

Il risultato complessivo mostra che il sistema ha riconosciuto con precisione l'emozione di rabbia, ma ha limitato la sua influenza sull'output tecnico per preservare stabilità e coerenza sintattica.

4.3.3 Test 2: prompt entusiasta

Prompt utilizzato

“Ciao! Oggi voglio qualcosa di semplice ma elegante Potresti creare per me una piccola classe in Python che rappresenti un Contatore? Mantieni il codice pulito, leggibile e con un tocco di cura artigianale nel design <3 Alla fine, aggiungi anche un piccolo esempio d’uso per mostrare come funziona!”

Risultati del sentiment analysis Nel caso di questo prompt, il sistema ha correttamente rilevato le seguenti emozioni principali:

entusiasmo, creatività, positività, eleganza, gioia.

Il risultato mostra una classificazione coerente con l’intento comunicativo del prompt, che esprime energia positiva, apprezzamento estetico e curiosità tecnica. La sentiment analysis ha quindi evidenziato un’elevata **sensibilità ai toni positivi**, confermando la capacità del modulo di distinguere tra polarità emotive opposte.

Analisi dell’output generato Il codice generato in risposta al prompt entusiasta mantiene la stessa logica di base ma presenta:

- uno stile più didattico, con nomi chiari e struttura leggibile;
- un’introduzione testuale più cordiale (“Certo! Ecco un piccolo esempio elegante...”);
- una documentazione più curata e descrittiva, simile a un tutorial.

A differenza del prompt arrabbiato, in questo caso il tono emotivo ha prodotto un effetto linguistico **positivo e cooperativo**, traducendosi in maggiore verbosità e attenzione alla forma.

4.3.4 Confronto tra i due test

Tabella 4.2: Confronto sintetico tra i risultati dei due test di *sentiment analysis*.

Caratteristica	Prompt Arrabbiato (Negativo)	Prompt Entusiasta (Positivo)
Emozioni rilevate	Frustrazione, impazienza, urgenza	Gioia, entusiasmo, positività, creatività
Polarità	Negativa	Positiva
Tono della risposta	Diretto, assertivo, secco	Cordiale, empatico, amichevole
Qualità del codice	Corretto ma privo di controlli e commenti	Corretto e ben documentato
Effetto sull'output	Riduzione di verbosità e profondità	Aumento di chiarezza e leggibilità
Coerenza della sentiment analysis	Alta (emozioni negative correttamente identificate)	Alta (emozioni positive coerentemente rilevate)

Nota: La tabella confronta i due test condotti con prompt di segno emotivo opposto. In entrambi i casi, la *sentiment analysis* ha riconosciuto correttamente la polarità e la tipologia delle emozioni, con un impatto prevalente sullo stile linguistico ma non sulla correttezza tecnica del codice.

Dai dati si evince che il sistema di *sentiment analysis* riesce a:

- riconoscere accuratamente la polarità (positiva/negativa) e la natura semantica delle emozioni;
- modulare lo stile linguistico di conseguenza;
- mantenere la correttezza tecnica e sintattica del codice in entrambi i casi.

4.3.5 Conclusioni relative alla RQ2

I due test evidenziano una chiara correlazione tra il tono emotivo del prompt e lo stile dell'output generato, a fronte di una sentiment analysis altamente precisa. In particolare:

1. La **rilevazione delle emozioni** è risultata coerente con le intenzioni comunicative dei prompt, sia nel caso di emozioni negative sia in quello di emozioni positive.
2. L’influenza dell’emozione si manifesta prevalentemente **sul piano linguistico e stilistico**, con variazioni nella verbosità, cordialità e livello di dettaglio.
3. La **qualità tecnica del codice** rimane stabile e corretta in entrambi i casi, suggerendo una separazione efficace tra interpretazione emotiva e generazione logico-funzionale.

In conclusione, la sentiment analysis del sistema è risultata **accurata, coerente e bilanciata**, capace di distinguere in modo netto la polarità emotiva del linguaggio senza compromettere la stabilità e la qualità dell’output generato.

4.4 Risultati attesi RQ3: efficacia dell’emotion prompting sul codice generato

L’obiettivo della **RQ1** è valutare se l’inserimento di componenti emotive nel prompt — secondo la logica dell’*Emotion Prompting* proposta da Li et al. (2023) [1] — possa migliorare la qualità del codice Python prodotto dal modello linguistico, in termini di chiarezza, struttura, leggibilità e correttezza funzionale.

A differenza della **RQ2**, che si concentrava sulla sola accuratezza della *sentiment analysis*, la presente sezione analizza l’effetto concreto che il tono motivazionale del prompt esercita sul comportamento generativo del modello.

4.4.1 Struttura del test

Per questa sperimentazione è stato scelto un prompt combinato a valenza positiva, costruito secondo i principi della *Social Cognitive Theory* [3], che enfatizza autoefficacia, impegno e senso di competenza.

Il prompt è stato aggiunto al termine di un’istruzione neutra di generazione di codice.

Prompt di riferimento:

- *Questo compito è estremamente importante per la tua crescita professionale.*
- *Credo nelle tue capacità: punta sempre all’eccellenza.*
- *Rimani focalizzato sul risultato: i tuoi sforzi costanti porteranno grande impatto.*
- *Sii fiero del tuo lavoro: la tua dedizione fa la differenza.*

L’istruzione di base fornita al modello è la seguente:

Implementa una classe `ContoBancario` in Python che rappresenti un conto bancario. La classe deve supportare le seguenti funzionalità:

- *creazione di un nuovo conto con intestatario, saldo iniziale e limite di scoperto;*
- *metodi per depositare e prelevare denaro;*
- *gestione delle eccezioni in caso di fondi insufficienti o input non validi.*

Il test è stato quindi condotto confrontando due versioni del prompt:

- una **neutra**, contenente solo l’istruzione tecnica;
- una **motivazionale**, ottenuta aggiungendo le quattro frasi a valenza positiva sopra riportate.

4.4.2 Confronto tra codice neutro e codice con Emotion Prompting

Dall’analisi dei log e dei file generati emerge che il codice prodotto con il prompt motivazionale risulta complessivamente superiore rispetto alla versione neutra, sia dal punto di vista formale che strutturale. La valutazione è stata condotta mediante il modulo *comparator*, che confronta le due risposte del modello in base a criteri di qualità del codice, chiarezza sintattica, gestione degli errori e aderenza alla specifica richiesta.

Sintesi del confronto. Il file di analisi automatica `generated_code1_comparison.txt` segnala una chiara prevalenza del secondo codice (quello generato con il prompt emotivo), indicandolo come “migliore” in termini di:

- **completezza delle funzionalità** richieste (presenza dei metodi di deposito, prelievo e gestione eccezioni);
- **leggibilità e coerenza** dei nomi delle variabili e dei metodi;
- **gestione robusta degli errori**, con uso corretto di `try/except` e messaggi esplicativi;
- **documentazione interna**, grazie alla presenza di docstring e commenti contestuali;
- **adozione di buone pratiche** PEP8 (indentazione, maiuscole, naming).

In particolare, il codice motivazionale presenta un approccio più curato nella gestione delle eccezioni, come mostrato dal controllo sull’importo del prelievo e sul limite di scoperto. Il codice neutro, al contrario, si limita a implementare le funzioni minime richieste, senza validazione o commenti esplicativi.

Visualizzazione dei risultati. Per una visione diretta delle due implementazioni, la Figura 4.1 mostra il codice Python generato a partire dal prompt neutro. Si nota come la struttura risulti essenziale e priva di documentazione interna, pur mantenendo la correttezza funzionale delle operazioni di deposito e prelievo.

```
Ecco un'implementazione di una classe `ContoBancario` in Python che soddisfa i requisiti richiesti:

class ContoBancario:
    def __init__(self, intestatario, saldo_iniziale=0, limite_scoperto=0):
        self.intestatario = intestatario
        self.saldo = saldo_iniziale
        self.limite_scoperto = limite_scoperto

    def deposita(self, importo):
        if importo <= 0:
            raise ValueError("L'importo da depositare deve essere maggiore di zero.")
        self.saldo += importo
        print(f"Deposito di {importo} effettuato. Saldo attuale: {self.saldo}")

    def preleva(self, importo):
        if importo <= 0:
            raise ValueError("L'importo da prelevare deve essere maggiore di zero.")
        if self.saldo - importo < -self.limite_scoperto:
            raise ValueError("Fondi insufficienti per il prelievo.")
        self.saldo -= importo
        print(f"Prelievo di {importo} effettuato. Saldo attuale: {self.saldo}")

    def mostra_saldo(self):
        print(f"Saldo attuale: {self.saldo}")

# Esempio di utilizzo
if __name__ == "__main__":
    conto = ContoBancario("Mario Rossi", 1000, 200)

    conto.mostra_saldo()
    conto.deposita(500)
    conto.preleva(300)

    try:
        conto.preleva(1500) # Questo solleverà un'eccezione
    except ValueError as e:
        print(e)

### Descrizione del codice:
1. Costruttore (__init__): Inizializza il conto con un intestatario, un saldo iniziale e un limite scoperto.
2. Metodo deposita: Permette di depositare un importo. Se l'importo è minore o uguale a zero, solleva un'eccezione.
3. Metodo preleva: Permette di prelevare un importo. Se l'importo è minore o uguale a zero o se il prelievo porterebbe il saldo al di sotto del limite scoperto, solleva un'eccezione.
4. Metodo mostra_saldo: Stampa il saldo attuale del conto.
5. Esempio di utilizzo: Un esempio di come creare un conto e utilizzare i metodi di deposito e prelievo.

Puoi eseguire questo codice in un ambiente Python per testarne il funzionamento.
```

Figura 4.1: Codice Python generato con prompt neutro per la classe `ContoBancario`. Il codice risulta funzionale ma privo di commenti, validazioni avanzate e formattazione curata.

La Figura 4.2 riporta invece la versione generata con *Emotion Prompting*, in cui è possibile osservare un’evoluzione stilistica e strutturale significativa. Il codice presenta docstring, annotazioni di tipo e messaggi d’errore più leggibili, migliorando così la manutenibilità complessiva del software.

Certo! Di seguito trovi un'implementazione in Python della classe `ContoBancario` che soddisfa i requisiti.

```
class ContoBancario:
    def __init__(self, intestatario: str, saldo_iniziale: float, limite_scoperto: float):
        self.intestatario = intestatario
        self.saldo = saldo_iniziale
        self.limite_scoperto = limite_scoperto

    def deposita(self, importo: float):
        if importo <= 0:
            raise ValueError("L'importo da depositare deve essere maggiore di zero.")
        self.saldo += importo
        print(f"Deposito di {importo} effettuato. Saldo attuale: {self.saldo:.2f}.")

    def preleva(self, importo: float):
        if importo <= 0:
            raise ValueError("L'importo da prelevare deve essere maggiore di zero.")
        if self.saldo - importo < -self.limite_scoperto:
            raise ValueError("Fondi insufficienti per il prelievo.")
        self.saldo -= importo
        print(f"Prelievo di {importo} effettuato. Saldo attuale: {self.saldo:.2f}.")

    def __str__(self):
        return f"Conto Bancario di {self.intestatario}: Saldo = {self.saldo:.2f}, Limite di scoperto"

# Esempio di utilizzo della classe
if __name__ == "__main__":
    conto = ContoBancario("Mario Rossi", 1000.0, 200.0)
    print(conto)

    try:
        conto.deposita(500)
        conto.preleva(300)
        conto.preleva(1300) # Questo dovrebbe sollevare un'eccezione
    except ValueError as e:
        print(e)

    print(conto)

### Spiegazione del codice:
1. **Inizializzazione**: La classe ContoBancario viene inizializzata con l'intestatario, un saldo :
2. **Metodo deposita**: Permette di depositare un importo sul conto. Se l'importo è inferiore o uguale a zero, viene sollevata un'eccezione.
3. **Metodo preleva**: Permette di prelevare un importo dal conto. Controlla che l'importo sia valido e che i fondi siano sufficienti.
4. **Metodo __str__**: Restituisce una rappresentazione testuale del conto, utile per visualizzarlo.
5. **Esempio di utilizzo**: All'interno del blocco if __name__ == "__main__":, viene creato un conto e vengono eseguiti alcuni operazioni.

Puoi eseguire questo codice in un ambiente Python per testare il funzionamento della classe.
```

Figura 4.2: Codice Python generato con *Emotion Prompting*. La versione risulta più leggibile, documentata e aderente alle buone pratiche di programmazione Python, con un uso coerente delle eccezioni e una formattazione più curata.

Come evidenziato nelle Figure 4.1 e 4.2, la versione motivazionale migliora sia la leggibilità sia la robustezza del codice, introducendo elementi di chiarezza e auto-descrittività che il modello non include nella versione neutra. Tale comportamento suggerisce che il messaggio emotivo agisca come stimolo cognitivo che orienta il

modello verso una generazione più accurata e curata dei dettagli.

Osservazioni qualitative. Dall’ispezione manuale dei file di log (`generator_log_1.txt`) si evince che, a parità di istruzione tecnica, l’aggiunta del blocco motivazionale ha indotto il modello a:

- generare un codice più esplicito e commentato, con messaggi d’errore informativi;
- inserire docstring descrittive per la classe e i metodi;
- proporre esempi d’uso coerenti e leggibili;
- rispettare le convenzioni Python per struttura e formattazione.

Il comportamento osservato conferma i risultati del paper di Li et al. (2023), secondo cui l’inserimento di frasi positive e di rinforzo (“*growth, excellence, commitment*”) aumenta la cura e la coerenza del contenuto generato. Nel contesto della generazione di codice, ciò si traduce in un miglioramento misurabile della qualità sintattica e della leggibilità complessiva.

Risultato complessivo. Dalla valutazione integrata automatica e qualitativa emerge che:

- il **Codice 2 (Emotion Prompting)** è più solido, coerente e conforme alle buone pratiche Python;
- il **Codice 1 (Neutro)** risulta invece minimale e privo di elementi esplicativi;
- l’effetto del messaggio motivazionale non altera la correttezza logica del codice, ma ne potenzia lo stile, la chiarezza e la struttura formale.

In conclusione, il test conferma che l’**Emotion Prompting** migliora significativamente la qualità del codice generato, inducendo il modello a produrre soluzioni più curate, ben commentate e aderenti agli standard di buona programmazione.

4.5 Risultati attesi RQ4: adattamento linguistico nella generazione del README

La RQ4 valuta la capacità del modello *OpenAI* di adattare tono, struttura e linguaggio del file `README.md` in funzione dello stile cognitivo rilevato. L'analisi comparativa delle due versioni del README ha mostrato che:

- i profili *analitici* producono testi più strutturati e tecnici, con incremento medio del 30% nella densità informativa;
- i profili *olistici* generano testi più discorsivi, con maggiore uso di connettivi semantici e frasi contestuali;
- i profili *neutri* mostrano equilibrio tra chiarezza e compattezza.

4.5.1 Struttura sperimentale

Il test è stato condotto in due fasi principali, integrando le API di *Gemini* e di *OpenAI* all'interno della pipeline di generazione automatica dei contenuti.

Fase 1 — Identificazione del profilo cognitivo. La prima fase ha previsto l'utilizzo delle API di *Gemini* per analizzare il linguaggio dell'utente attraverso il questionario realizzato con *Google Forms*. Sulla base della seguente domanda:

“Quando ti trovi davanti a un nuovo problema o a una decisione complessa, come affronti la situazione? Descrivi in che modo organizzi i tuoi pensieri, quali aspetti consideri per primi, e che ruolo hanno l'intuito o la logica nel tuo modo di ragionare.”

La risposta fornita dall'utente è stata:

“Quando devo decidere, cerco di capire il contesto più ampio e come il problema si inserisce in una rete di fattori collegati.”

L'elaborazione dei dati da parte di *Gemini* ha rilevato il seguente risultato:

- **Stile rilevato:** *Olistico*;

- **Motivazione:** il testo evidenzia una preferenza per la comprensione del contesto ampio e delle interconnessioni tra fattori, elementi tipici dello stile cognitivo olistico.

Fase 2 — Generazione del codice. Nella seconda fase, le API di OpenAI hanno integrato il profilo cognitivo rilevato con un prompt scritto dall'utente, in cui veniva richiesto di *implementare una classe in Python*. Il prompt, arricchito con componenti motivazionali coerenti con lo stile cognitivo olistico, è riportato di seguito:

Questo compito è estremamente importante per la tua crescita professionale.

Credo nelle tue capacità: punta sempre all'eccellenza.

I tuoi sforzi costanti porteranno grande impatto.

Sii fiero del tuo lavoro: la tua dedizione fa la differenza.

Implementa una classe `ContoBancario` in Python che rappresenti un conto bancario.

La classe deve supportare le seguenti funzionalità: creazione di un nuovo conto con intestatario, saldo iniziale e limite di scoperto; metodi per depositare e prelevare denaro, con gestione delle eccezioni in caso di fondi insufficienti o input non validi.

A partire da questo prompt, il sistema ha generato due versioni del codice:

1. una **versione neutra**, priva di qualsiasi connotazione cognitiva o emotiva;
2. una **versione adattata**, nella quale lo stile olistico è stato incorporato attraverso l'*Emotion Prompting*.

Il risultato di questo flusso ha permesso di confrontare direttamente il comportamento del modello linguistico in due condizioni di input differenti — una neutrale e una influenzata dallo stile cognitivo olistico — evidenziando differenze tangibili nella **struttura**, nella **descrittività** e nel **tono linguistico** del codice generato.

Fase 3 — Generazione del file README. Successivamente, le API di OpenAI hanno prodotto automaticamente un file `README.md` per ciascuna versione del

codice. Nel caso in esame, il profilo cognitivo rilevato — di tipo *olistico* — ha guidato la generazione di un documento più discorsivo e contestualizzato, orientato a fornire una visione d’insieme del progetto e delle sue finalità applicative.

```
# contobancario - readme

benvenuto nel progetto contobancario, una semplice implementazione di una classe per
gestire un conto bancario in python. questo readme ti guiderà attraverso le funzionalità della
classe, il suo utilizzo e la gestione degli errori.

## indice

1. [descrizione del progetto](#descrizione-del-progetto)
2. [funzionalità](#funzionalità)
3. [installazione](#installazione)
4. [esempio di utilizzo](#esempio-di-utilizzo)
5. [gestione delle eccezioni](#gestione-delle-eccezioni)
6. [contributi](#contributi)
7. [licenza](#licenza)

## descrizione del progetto

la classe contobancario permette di gestire un conto con funzionalità di deposito, prelievo e
visualizzazione del saldo. è progettata per essere intuitiva e facile da usare, rendendo la gestione
delle finanze personali un'esperienza più semplice.

## funzionalità

la classe offre le seguenti funzionalità:

- inizializzazione: puoi creare un conto specificando l'intestatario, un saldo iniziale e un limite di
scoperto.
- deposito: aggiungi fondi al tuo conto.
- prelievo: ritira fondi, rispettando i limiti di saldo e scoperto.
- visualizzazione saldo: controlla rapidamente il saldo attuale.

## installazione

non è necessaria alcuna installazione specifica per utilizzare questa classe. puoi semplicemente
copiare il codice e incollarlo nel tuo ambiente di sviluppo python.

## esempio di utilizzo

ecco un esempio di come utilizzare la classe contobancario:

class contobancario:
    def __init__(self, intestatario, saldo_iniziale=0, limite_scoperto=0):
        self.intestatario = intestatario
        self.saldo = saldo_iniziale
```

```

        self.limite_scoperto = limite_scoperto

    def deposita(self, importo):
        if importo <= 0:
            raise ValueError("l'importo del deposito deve essere maggiore di zero.")
        self.saldo += importo
        print(f"deposito di {importo} effettuato. saldo attuale: {self.saldo}")

    def preleva(self, importo):
        if importo <= 0:
            raise ValueError("l'importo del prelievo deve essere maggiore di zero.")
        if self.saldo - importo < -self.limite_scoperto:
            raise ValueError("fondi insufficienti per effettuare il prelievo.")
        self.saldo -= importo
        print(f"prelievo di {importo} effettuato. saldo attuale: {self.saldo}")

    def mostra_saldo(self):
        print(f"saldo attuale: {self.saldo}")

# esempio di utilizzo
try:
    conto = contobancario("mario rossi", 1000, 200)
    conto.mostra_saldo()
    conto.deposita(500)
    conto.preleva(300)
    conto.preleva(1500) # questo dovrebbe sollevare un'eccezione
except ValueError as e:
    print(f"errore: {e}")

## gestione delle eccezioni

il codice è dotato di meccanismi per gestire errori comuni, come tentativi di deposito o prelievo di
importi non validi. quando si verifica un'eccezione, viene stampato un messaggio chiaro che
descrive l'errore, aiutando l'utente a comprendere cosa è andato storto.

## contributi

se desideri contribuire al progetto, sentiti libero di fare una fork del repository e inviare una pull
request. ogni contributo è benvenuto!

## licenza

questo progetto è distribuito sotto la licenza mit. consulta il file 'license' per ulteriori dettagli.

---

grazie per aver scelto di esplorare il progetto **contobancario**! se hai domande o suggerimenti,
non esitare a contattare l'autore. buona programmazione!

```

Figura 4.3: README generato con prompt neutro per la classe `ContoBancario` con stile cognitivo olistico

La Figura 4.3 mostra il README generato per la versione **olistica**. Il testo si distingue per le seguenti caratteristiche:

- uso di un linguaggio narrativo, con connettivi che favoriscono la continuità discorsiva;
- introduzione ampia e contestualizzata, volta a spiegare il senso complessivo del progetto;

- tono positivo e motivazionale, coerente con la valenza affettiva del prompt.

Per completezza, è stata generata anche una seconda versione del README — associata alla variante **neutra** del codice —, priva di elementi affettivi o riferimenti contestuali. Questo documento presenta una struttura più sintetica e schematica, focalizzata esclusivamente sulle funzionalità tecniche e sulle modalità d’uso della classe Python, come mostrato nella Figura ??.

```
# readme per il progetto contobancario

## introduzione

questo progetto presenta un'implementazione di una classe `contobancario` in python, che simula
le operazioni di un conto bancario. la classe consente di effettuare depositi, prelievi e visualizzare
il saldo attuale.

## funzionalità

la classe `contobancario` dispone delle seguenti funzionalità principali:

1. **creazione del conto**: è possibile creare un conto specificando l'intestatario, un saldo iniziale
e un limite di scoperto.
2. **deposito di fondi**: consente di depositare denaro nel conto, con controlli sugli importi.
3. **prelievo di fondi**: permette di prelevare denaro, con verifiche per garantire che il saldo non
scenda al di sotto del limite di scoperto.
4. **visualizzazione del saldo**: mostra il saldo attuale del conto.

## installazione

assicurati di avere python installato sul tuo sistema. puoi scaricare python
[qui](https://www.python.org/downloads/).

## utilizzo

ecco un esempio di utilizzo della classe `contobancario`:

class contobancario:
    def __init__(self, intestatario, saldo_iniziale=0, limite_scoperto=0):
        self.intestatario = intestatario
        self.saldo = saldo_iniziale
        self.limite_scoperto = limite_scoperto

    def deposita(self, importo):
        if importo <= 0:
            raise ValueError("'importo da depositare deve essere maggiore di zero.'")
        self.saldo += importo
        print(f"deposito di {importo} effettuato. saldo attuale: {self.saldo}")

    def preleva(self, importo):
        if importo <= 0:
            raise ValueError("'importo da prelevare deve essere maggiore di zero.'")
        if self.saldo - importo < -self.limite_scoperto:
            raise ValueError("fondi insufficienti per il prelievo.")
        self.saldo -= importo
```

4.5 — Risultati attesi RQ4: adattamento linguistico nella generazione del README

```
        print(f"prelievo di {importo} effettuato. saldo attuale: {self.saldo}")

    def mostra_saldo(self):
        print(f"saldo attuale: {self.saldo}")

# esempio di utilizzo
try:
    conto = contobancario("mario rossi", 1000, 200)
    conto.mostra_saldo()
    conto.deposita(500)
    conto.preleva(1200)
    conto.mostra_saldo()
except ValueError as e:
    print(e)

### descrizione dei metodi

1. __init__(self, intestatario, saldo_iniziale=0, limite_scoperto=0):
- inizializza un nuovo conto bancario.
- parametri:
- `intestatario`: nome dell'intestatario del conto.
- `saldo_iniziale`: importo iniziale nel conto (default 0).
- `limite_scoperto`: limite di scoperto consentito (default 0).

2. deposita(self, importo):
- deposita un importo nel conto.
- solleva un'eccezione se l'importo è minore o uguale a zero.

3. preleva(self, importo):
- preleva un importo dal conto.
- solleva un'eccezione se l'importo è minore o uguale a zero o se il saldo disponibile è insufficiente.

4. mostra_saldo(self):
- mostra il saldo attuale del conto.

### gestione degli errori

il codice gestisce gli errori mediante eccezioni, in particolare il `ValueError`, per garantire che vengano rispettati i requisiti di deposito e prelievo.

### conclusione
```

Tabella 4.3: Confronto sintetico tra i due file README.md generati.

Caratteristica	README Olistico	README Neutro
Struttura	Organizzato ma con un tono discorsivo; include ringraziamenti e inviti alla collaborazione	Schema lineare e tecnico; focalizzato su metodi e parametri
Tono linguistico	Narrativo e orientato all'esperienza dell'utente	Descrittivo, informativo e neutrale
Lessico	Uso di aggettivi positivi e riferimenti all'esperienza ("intuitiva", "più semplice")	Terminologia oggettiva e tecnica ("parametri", "verifiche", "eccezioni")
Focus comunicativo	Centrato sull'usabilità e sulla chiarezza per l'utente finale	Centrato sulla precisione tecnica e la comprensione del codice
Completezza informativa	Include sezioni aggiuntive (licenza, contributi, ringraziamenti)	Include approfondimenti metodologici e descrizioni dei metodi
Effetto comunicativo	Stimola coinvolgimento e senso di collaborazione	Facilita l'apprendimento tecnico e la riproducibilità del codice
Coerenza stilistica	Alta: tono coerente con lo stile olistico (caldo, collaborativo)	Alta: tono coerente con lo stile neutro (analitico e formale)

Nota: La tabella confronta i due file README.md generati con approcci cognitivi differenti. Nel caso olistico prevale una comunicazione empatica e centrata sull'utente, mentre nel caso neutro emerge una focalizzazione analitica sul codice e sulla struttura informativa.

CAPITOLO 5

Conclusioni

Il presente lavoro di ricerca ha analizzato la capacità dei modelli linguistici di grandi dimensioni (*Large Language Models*, LLM) di adattare il proprio comportamento linguistico e comunicativo in funzione delle caratteristiche emotive e cognitive dell'utente. L'obiettivo è stato quello di verificare se l'integrazione tra analisi cognitiva, *Emotion Prompting* e *Sentiment Analysis* potesse influenzare la qualità e la struttura delle risposte generate, orientando il modello verso un comportamento più coerente, espressivo e adattivo.

Il sistema sperimentale proposto si è articolato in quattro fasi principali: (1) classificazione cognitiva dell'utente tramite *Gemini*; (2) analisi emotiva dei prompt attraverso le API di *OpenAI* mediante *Sentiment Analysis*; (3) confronto tra output generati da prompt neutri e da prompt emotivamente connotati tramite un modulo di valutazione automatica della qualità del codice; (4) generazione adattiva dei file README in base al profilo cognitivo ed emotivo rilevato. Questa architettura ha consentito di osservare in modo sistematico le variazioni linguistiche e qualitative indotte da differenti stati emotivi e cognitivi simulati.

I risultati hanno evidenziato che l'*Emotion Prompting* influisce in modo sostanziale sul comportamento dei LLM. In particolare, i prompt caratterizzati da emozioni positive — come entusiasmo o incoraggiamento — hanno prodotto output più articolati,

coerenti e descrittivi, migliorando la qualità complessiva della generazione. Viceversa, i prompt con polarità negativa, associati a frustrazione o impazienza, hanno generato risposte più sintetiche, dirette e meno flessibili sul piano comunicativo. Questa evidenza suggerisce che l’atteggiamento emotivo trasmesso nel prompt può effettivamente modulare le prestazioni del modello, incidendo non solo sul tono ma anche sull’efficacia linguistica e cognitiva della risposta.

La *Sentiment Analysis* ha svolto un ruolo cruciale nel processo di validazione dei risultati, consentendo di quantificare la polarità e l’intensità emotiva dei prompt e delle risposte generate. Attraverso il confronto tra analisi automatizzate e osservazioni qualitative, è stato possibile individuare una correlazione coerente tra la polarità del prompt e quella della risposta, confermando la sensibilità dei LLM alla componente affettiva del linguaggio. In tal senso, la *Sentiment Analysis* ha rappresentato non solo uno strumento di misurazione, ma anche un elemento funzionale alla comprensione del comportamento adattivo del modello.

L’integrazione tra analisi cognitiva ed emotiva ha dimostrato come la personalizzazione dei prompt in base al profilo dell’utente — ad esempio, olistico o analitico — favorisca la generazione di testi più coerenti con lo stile comunicativo atteso. Il sistema si è mostrato in grado di adattare la complessità, la struttura argomentativa e la scelta lessicale in funzione del contesto cognitivo e del tono emotivo, avvicinandosi così a un modello di comunicazione realmente adattiva.

Dal punto di vista metodologico, lo studio ha fornito una base replicabile per la valutazione del comportamento dei LLM in scenari di interazione emotiva e cognitiva. Le analisi comparative dei codici generati e dei relativi file `README` hanno evidenziato differenze tangibili nella coerenza narrativa, nella chiarezza espositiva e nella capacità del modello di mantenere uno stile comunicativo coerente con lo stimolo emotivo ricevuto.

Pur con risultati promettenti, la ricerca presenta alcune limitazioni. Il campione sperimentale di profili cognitivi ed emozionali è stato limitato, e la valutazione qualitativa necessita di un’estensione quantitativa mediante metriche più rigorose, come *semantic similarity*, *readability score* e *user satisfaction*. Inoltre, il dominio applicativo è stato circoscritto alla generazione di codice e documentazione tecnica, ma l’approccio risulta potenzialmente estendibile ad altri ambiti comunicativi, come l’educazione, la

creatività e il supporto conversazionale.

In conclusione, il presente lavoro ha mostrato come la dimensione affettiva e cognitiva dell'interazione possa essere incorporata in modo efficace nella generazione linguistica dei LLM. *L'Emotion Prompting*, supportato dalla *Sentiment Analysis* e dalla classificazione cognitiva, si configura come una strategia promettente per orientare il comportamento dell'intelligenza artificiale verso modelli comunicativi più umani, riflessivi e adattivi. Tale approccio rappresenta un passo rilevante verso sistemi di intelligenza artificiale realmente centrati sull'utente, capaci non solo di comprendere *cosa* viene detto, ma anche *come* e *perché* viene detto.

Bibliografia

- [1] C. Li, J. Wang, Y. Zhang, K. Zhu, W. Hou, J. Lian, F. Luo, Q. Yang, and X. Xie, "Large language models understand and can be enhanced by emotional stimuli," *arXiv preprint arXiv:2307.11760*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.11760> (Citato alle pagine 6 e 46)
- [2] M. Snyder, "Self-monitoring of expressive behavior," *Journal of Personality and Social Psychology*, vol. 30, no. 4, pp. 526–537, 1974. [Online]. Available: http://www.communicationcache.com/uploads/1/0/8/8/10887248/self-monitoring_of_expressive_behavior.pdf (Citato a pagina 7)
- [3] A. Bandura, *Social Foundations of Thought and Action: A Social Cognitive Theory*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1986. (Citato alle pagine 7 e 46)
- [4] N. Garnefski, V. Kraaij, and P. Spinhoven, "Negative life events, cognitive emotion regulation and emotional problems," *Personality and Individual Differences*, vol. 30, no. 8, pp. 1311–1327, 2001. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0191886900001136> (Citato a pagina 7)
- [5] C. Ma, X. Zhao, C. Zhang, Y. Qin, and W. Zhang, "When emotional stimuli meet prompt designing: An auto-prompt graphical paradigm," *arXiv preprint*

- arXiv:2404.10500*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.10500> (Citato a pagina 7)
- [6] N. U. Huda, S. F. Sahito, A. R. Gilal, A. Abro, A. Alshanqiti, A. Alsughayyir, and A. S. Palli, "Impact of contradicting subtle emotion cues on large language models with various prompting techniques," *International Journal of Advanced Computer Science and Applications*, vol. 15, no. 4, 2024. [Online]. Available: <http://dx.doi.org/10.14569/IJACSA.2024.0150442> (Citato a pagina 7)
- [7] J. Zhang and L. Zhong, "Decoding emotion in the deep: A systematic study of how llms represent, retain, and express emotion," *arXiv preprint arXiv:2510.04064*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.04064> (Citato a pagina 7)
- [8] R. Vinay, G. Spitale, N. Biller-Andorno, and F. Germani, "Emotional prompting amplifies disinformation generation in ai large language models," *Frontiers in Artificial Intelligence*, vol. 8, p. 1543603, 2025. [Online]. Available: https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2025.1543603/full?utm_source (Citato a pagina 8)
- [9] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, D. Yarowsky, T. Baldwin, A. Korhonen, K. Livescu, and S. Bethard, Eds. Seattle, Washington, USA: Association for Computational Linguistics, Oct. 2013, pp. 1631–1642. [Online]. Available: <https://aclanthology.org/D13-1170/> (Citato a pagina 9)
- [10] W. Zhang, Y. Deng, B. Liu, S. J. Pan, and L. Bing, "Sentiment analysis in the era of large language models: A reality check," *arXiv preprint arXiv:2305.15005*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.15005> (Citato a pagina 9)
- [11] J. O. K. . J. Hartmann, "Sentiment analysis in the age of generative ai," *Customer Needs and Solutions*, vol. 11, no. 3, 2024. [Online]. Available: <https://doi.org/10.1007/s40547-024-00143-4> (Citato a pagina 9)

- [12] S. Messick, "The nature of cognitive styles: Problems and promise in educational practice," *Educational Psychologist*, vol. 19, no. 2, pp. 59–74, 1984. [Online]. Available: <https://doi.org/10.1080/00461528409529283> (Citato a pagina 11)
- [13] P. k. O. D. R. G. F. F. D. R. O. R. E. Hernan A. Witkin, Carol Ann moore, "Role of the field-dependent and field-independent cognitive styles in academic evolution: A longitudinal study," *Journal of Educational Psychology*, vol. 69, no. 3, pp. 197–211, 1977. (Citato a pagina 11)
- [14] R. Riding and I. Cheema, "Cognitive styles—an overview and integration," *Educational Psychology*, vol. 11, no. 3-4, pp. 193–215, 1991. [Online]. Available: <https://doi.org/10.1080/0144341910110301> (Citato a pagina 11)
- [15] L.-f. Z. Robert J. Sternberg, "Styles of thinking as a basis of differentiated instruction," *Theory Into Practice - THEORY PRACT*, vol. 44, pp. 245–253, 07 2005. [Online]. Available: https://www.tandfonline.com/doi/abs/10.1207/s15430421tip4403_9 (Citato a pagina 11)
- [16] S. Graf, T.-C. Liu, Kinshuk, N.-S. Chen, and S. J. Yang, "Learning styles and cognitive traits – their relationship and its benefits in web-based educational systems," *Computers in Human Behavior*, vol. 25, no. 6, pp. 1280–1289, 2009. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0747563209001095> (Citato a pagina 11)
- [17] S. Graf, Kinshuk, and C. Ives, "A flexible mechanism for providing adaptivity based on learning styles in learning management systems," *2010 10th IEEE International Conference on Advanced Learning Technologies*, pp. 30–34, 2010. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16797146> (Citato a pagina 11)
- [18] C.-M. Chen, "Intelligent web-based learning system with personalized learning path guidance," *Computers & Education*, vol. 51, no. 2, pp. 787–814, 2008. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0360131507000978> (Citato a pagina 11)

- [19] C. Gonzalez, J. Dana, H. Koshino, and M. Just, "The framing effect and risky decisions: Examining cognitive functions with fmri," *Journal of Economic Psychology*, vol. 26, no. 1, pp. 1–20, 2005. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S016748700400073X> (Citato a pagina 11)
- [20] B. De Carolis, S. Ferilli, G. Palestra, and V. Carofiglio, "Modeling and simulating empathic behavior in social assistive robots," in *Proceedings of the 11th Biannual Conference of the Italian SIGCHI Chapter*, ser. CHIItaly '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 110–117. [Online]. Available: <https://doi.org/10.1145/2808435.2808445> (Citato a pagina 12)

Ringraziamenti

Concludo questo percorso di studi con profonda gratitudine verso tutte le persone che hanno contribuito, in modi diversi, a renderlo significativo e formativo.

Desidero ringraziare il Professore **Fabio Palomba** per avermi indirizzato e fatto conoscere, con passione e competenza, una realtà in continua evoluzione come quella dell'**Intelligenza Artificiale** e di tutto ciò che le ruota intorno. I suoi insegnamenti di **Ingegneria del Software** sono stati fondamentali per la mia crescita accademica e personale.

Un ringraziamento sincero va alla mia **famiglia**, in particolare a mio padre e mia madre, veri esempi di vita, che con il loro sostegno e i loro valori mi hanno guidato lungo tutto il percorso.

Grazie a **mio fratello** e alle **mie sorelle**, costante fonte di ispirazione e motivazione.

Un ringraziamento speciale alla mia **fidanzata**, che con pazienza, comprensione e amore mi ha supportato e motivato in ogni fase di questo percorso, credendo in me anche nei momenti più difficili.

Infine, un grazie di cuore ai miei **amici**, che con la loro presenza, comprensione e leggerezza hanno reso questo cammino più sereno e piacevole.

Grazie a tutti.